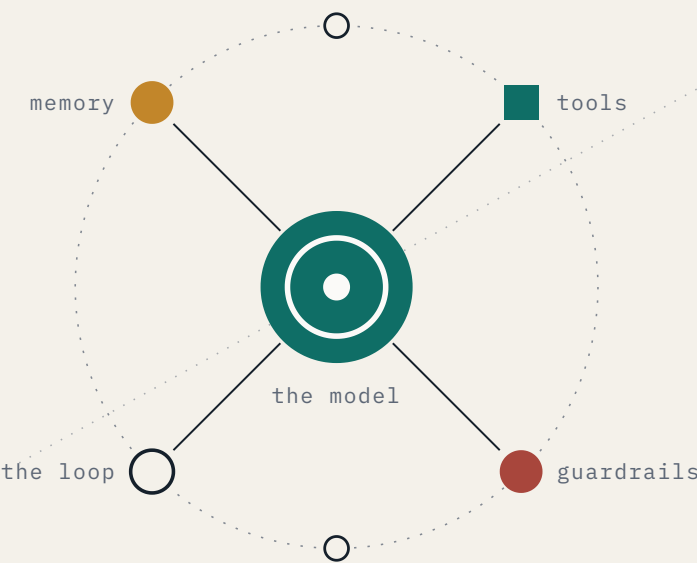


The Curious Mind's Guide to Agentic AI

*How the machines reshaping work actually think —
from the next word to autonomous agents.*



Gagan Sachdeva

I-IV • THE MIND

V-VI • THE HANDS

VII • THE ENGINE

The Curious Mind's Guide to Agentic AI

THE PHILOSOPHICAL LEDGER

The Curious Mind's Guide to Agentic AI

How the machines reshaping work actually think — from the next word to autonomous agents.

Gagan Sachdeva

A FIELD GUIDE IN SEVEN PARTS · gagansachdeva.com

The Curious Mind's Guide to Agentic AI · Special Edition

A publication of The Philosophical Ledger · gagansachdeva.com

Written and designed by Gagan Sachdeva.

Rebuilt for the non-technical reader from the technical literature on large language models and agentic AI.

Set in **Fraunces**, **Spectral** & **IBM Plex Mono**.

Every diagram drawn from first principles. No equations in the main text.

This work is offered freely to readers as a special piece.

Please share it; please credit it.

FOREWORD

A note before we begin

I spend my days building AI systems inside a large bank — the governed, accountable, unglamorous kind of AI that has to actually work when real money and real trust are on the line. The deeper I went, the more I noticed the same thing everywhere: a widening gap between the few who understand how these systems work and the many who must live with the consequences of them. That gap is where bad decisions hide.

This guide is my attempt to close a little of it. I took the genuine technical literature — dense, mathematical, and largely closed to outsiders — and rebuilt its real ideas in language any curious person can hold, without watering them down. It follows the editorial frame I write under at The Philosophical Ledger: *signal, meaning, action*. Find the signal beneath the noise; make its meaning plain; leave you able to act.

It is offered freely, as a special piece, to anyone who would rather understand than be impressed. If it helps you see these machines a little more clearly — and trust them a little more wisely — it will have done its job.

— Gagan Sachdeva

Adapted, with gratitude, from the open technical reference
“The Hitchhiker's Guide to Agentic AI” by Haggai Roitman — rebuilt for the non-specialist reader.

“You won’t be impressed by AI the way a person is impressed by a magic trick. You’ll be impressed the way an engineer is impressed by a bridge — because you can see how it holds itself up, and exactly where it would fall down.”

FROM THE INTRODUCTION

HOW TO READ THIS BOOK

Why this guide exists

There is a strange gap opening up in the world. On one side are a few thousand people who build these systems; to them, the behaviour of an AI is not magic — it is mechanism. They know *why* it makes things up, *why* it's confident when it's wrong, *why* it's brilliant at some tasks and hopeless at others that look easier. On the other side is nearly everyone else — including people making large, consequential decisions about tools they can describe but not explain.

This guide is for the second group — for the ones who refuse to stay there. The serious technical literature is extraordinary and almost entirely closed to outsiders: the reference this series is built from runs to six hundred pages of equations, code, and hardware diagrams. What I've done is take the *real* ideas inside it — not a watered-down cartoon, the actual load-bearing concepts — and rebuild them in language you can hold, with the maths taken off so you can pick them up.

Read it in order. Each part assumes the one before it, the diagrams carry as much of the explanation as the words, and every part closes with the honest questions you'd still be asking. Where the precise machinery behind a metaphor is worth having, a short *Going deeper* note waits at the end of each part; it's optional.

FIRST, THE WORD IN THE TITLE

What do we mean by “agentic AI”?

Almost every AI you've used so far *answers*. You ask; it replies; you're done. An **agent** is the next thing: an AI that can take actions to accomplish a goal — use tools, search the web, run code, fill in forms, call other software, work through many steps, and check its own progress — with only light steering from you.

The difference is the difference between advice and action. Ask a normal assistant “*how do I book a flight to Tokyo?*” and it tells you the steps. Ask an *agent* and it opens the site, compares the options, fills in your details, and books it. That loop — *decide, act, observe, decide again* — is the whole idea. The model stops being a clever oracle and becomes a worker.

So why spend four parts on the underlying model before we reach the agents? Because **an agent is not a different kind of machine — it's everything in this guide, wired together and pointed at a goal**. At its core sits a language model (Part I), taught to be helpful (Part II), able to reason (Part III), and only trustworthy insofar as we can measure it (Part IV). Bolt that mind to tools, a

memory, and a loop, and you have an agent (Parts V–VI). You cannot understand — or trust, or govern — the agent until you understand the mind inside it.

Parts I–IV are the mind; Parts V–VI are the hands and the world; Part VII is the engineering that holds it all up. Wherever you are in the climb, that's the map.

CONTENTS

The whole map

- Introduction & how to read this book
-

Part I • The Curious Mind's Guide to Agentic AI

How These Machines Actually Work

Preface Why this book exists

One The Machine That Only Predicts the Next Word

Two First, It Breaks Your Words Apart

Three How It Pays Attention

Four How It Decides What to Say

Five The Machine Underneath the Machine

Part II • Teaching a Mind Its Manners

How a Model Learns Manners & Judgment

One The Three Stages of Raising a Model

Two Reading the Whole World

Three Teaching by Example

Four Learning From What People Prefer

Five The Shortcut, and What It Costs

Part III • The Pause Before the Answer

How a Model Learns to Think Before It Speaks

- One** The Blurt
 - Two** Thinking Out Loud
 - Three** Discovering Reasoning
 - Four** Checking the Work
 - Five** How Much Thinking Is Worth It
-

Part IV • Grading the Ungradeable

How We Know Any of It Works

- One** Why Measuring Is Hard
 - Two** Catching the Confident Fabrication
 - Three** The Judge Is Also a Machine
 - Four** Benchmarks and Their Lies
 - Five** Grading an Agent
-

Part V • Giving the Machine Hands

Giving the Machine Hands

- One** From Answering to Acting
 - Two** Looking Things Up
 - Three** Remembering
 - Four** The Harness
 - Five** Patterns That Work
-

Part VI • A World of Cooperating Agents

A World of Cooperating Agents

- One** Plugs and Sockets
- Two** Teaching an Agent New Tricks

Three Agents Talking to Agents

Four The Org Chart of Machines

Five Building and Showing

Part VII • Under the Hood

How They Made It Affordable

One A Panel of Specialists

Two Shrinking Without Lobotomising

Three The Fast Intern

Four Keeping the Hands Fed

Five The Data-Centre Reality

Afterword • So What Now?

What It Means, and How to Use It Well

One The line that actually matters

Two What becomes more valuable, not less

Three How to actually get good at using them

Four If the thing adopting this is your workplace

Reference

The Road Ahead, Self-Tests & Glossary

One Where This Is Going

Two Check Your Understanding

Three The Glossary

PART I

The Curious Mind's Guide to Agentic AI

How the machines that are reshaping work actually think — explained for people who don't build them, but refuse to be fooled by them.

How These Machines Actually Work

How to read this book. Read it in order — each idea is built to stand on the one before it, so the climb only works from the bottom. The diagrams carry as much of the explanation as the words, so linger on them. And if you ever want the precise machinery behind a metaphor, a *Going deeper* note waits at the back of each set.

This is **Set 1** of a longer series. It covers the foundation everything else stands on: what a large language model is, how it reads, how it pays attention, how it writes — and the physical machine all of that runs on. We go one floor at a time, and never pretend a floor exists before we've built it.

PREFACE

Why this book exists

There is a strange gap opening up in the world. On one side are a few thousand people who build these systems; to them, the behaviour of an AI is not magic — it is mechanism. On the other side is nearly everyone else, including people making large, consequential decisions about tools they can describe but not explain.

This book is for the second group — specifically, the ones who refuse to stay there. The technical literature on this subject is extraordinary and almost entirely closed to you: the reference I'm translating from runs to six hundred pages of equations, code, and hardware diagrams, written for the people who build. What I want to do is take the *real* ideas inside it — not a watered-down cartoon, the actual load-bearing concepts — and rebuild them in language you can hold. Semi-technical, never *non*-technical. I'm not going to protect you from the ideas. I'm going to hand them to you with the math taken off, so you can pick them up.

By the end, you will not be impressed by AI the way a person is impressed by a magic trick. You will be impressed the way an engineer is impressed by a bridge — because you can see how it holds itself up, and exactly where it would fall down.

That second part matters more than the first.

The journey ahead

The full road has five stages. You don't need to memorise them; this is just the shape of the thing.

- **Foundations — how the machine works.** (*This set.*) What a model is, how it reads, attends, writes, and what it physically runs on. The bedrock.
- **Teaching — how it learns manners and judgment.** A raw model is fluent but feral. We'll see how it's turned into something helpful and steerable — and what gets traded away.

- **Reasoning** — **how it learns to think before it speaks.** The newest leap: models that work a problem step by step, an ability that was *discovered* more than designed.
- **Measuring** — **how we know any of it works.** The unglamorous discipline of telling a capable system from a merely convincing one.
- **Agents** — **how it gets hands.** The frontier: models that don't just answer but *act* — using tools, remembering, coordinating. Where the stakes get real.

Each stage assumes the one before. The single biggest reason people misunderstand AI is that they start in the middle — they meet an "agent" before they understand it's a next-word predictor wearing a tool-belt. We start at the bottom.



CHAPTER ONE

The Machine That Only Predicts the Next Word

Here is the most important sentence in this book, and I want you to distrust it immediately:

A large language model is a machine that, given some text, predicts the next small piece of text — and then does it again, and again, until it decides to stop.

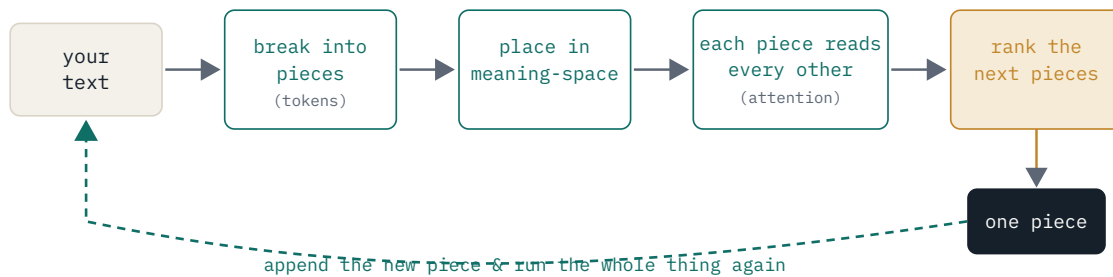
That's the whole engine. Not a database of facts. Not a reasoning system someone designed. Not a search engine. A *next-piece predictor*, run in a loop.

Why distrust it? Because it's true the way "a brain is just cells passing chemicals" is true — accurate, and almost useless on its own. The interesting part is what happens when you do that one simple thing at a scale that beggars intuition. So let's earn the understatement back.

The pipeline, at altitude

Whenever you type something and the system responds, your text makes the same journey, every time, in the same order: **text** → **pieces** → **meanings** → **a prediction** → **text again**.

FIG 1 the pipeline – and the loop that is its secret



*The model never writes a sentence. It writes one piece, re-reads everything — including what it just wrote — and writes the next. A whole essay is this loop spinning hundreds of times. The technical name is *autoregression*: it eats its own output.*

Where does the "knowing" come from?

If it's just predicting the next piece, why does it seem to *know things*? Because of how it learned to predict. Before you ever met it, the model was shown an almost unimaginable amount of text — a meaningful fraction of everything humans have written down — and set one brutally simple exercise, billions of times: *here is some text with the next piece hidden; guess it*.

At first it guesses nonsense. But every wrong guess nudges its internal settings a hair toward being right next time. Do this enough and something remarkable happens. To get genuinely good at guessing the next word in *human writing*, the machine is quietly forced to absorb the *patterns* of that writing — and our writing is soaked in our knowledge. You cannot reliably finish "The capital of France is" without, in some functional sense, having captured that it's Paris. The knowledge isn't stored as facts in a table; it's compressed into the prediction machinery itself, as a side effect of mastering one narrow game. The understanding is *real enough to be useful* and *not the kind you have* — and holding both thoughts at once is the beginning of wisdom about these systems.

The honest part: this is also why it lies

Now we collect the bill. The model is not trying to tell you the truth. It has never, at any point, been built to. It is trying to produce a *plausible continuation* — the piece that *fits*. For well-trodden topics, the most plausible continuation usually is the true one, because true things were written down most often. Plausibility and truth run side by side and mostly agree.

But push it somewhere thinly documented — an obscure person, a niche legal point, a paper that doesn't exist — and the two part ways. The model still produces something plausible, fluent, and confident, because that's the *only* thing it can do. We call this a *hallucination*, but the word makes it sound like a malfunction. It isn't. It's the machine working exactly as designed, in a place where "plausible" and "true" have quietly diverged and no one told it.

Fluency and accuracy are produced by the same mechanism, and that mechanism optimises for fluency.

Hold onto this. Everything in the later sets — the teaching, the reasoning, the tools, the evaluation — is in large part an effort to drag accuracy back into line with fluency, because the base machine will not do it for free.

IN THE WILD

The assistants you already use — **ChatGPT, Claude, Gemini, Copilot** — are all this same next-word machinery, just raised and dressed differently. Nothing in this book is about one company's secret sauce; it's about the shared mechanism underneath all of them.

RECAP

A model predicts the next small piece of text and loops. Its apparent knowledge is the residue of mastering that game across nearly all human writing. Its tendency to fabricate is the same engine running where plausibility and truth disagree. Next: what those "small pieces" actually are.

CHAPTER TWO

First, It Breaks Your Words Apart

Before the machine can do anything clever, it has to chop your sentence up. The way it chops — into units called **tokens** — quietly explains a surprising amount of AI's everyday weirdness: why it's worse in some languages, why it sometimes can't count the letters in a word, why long documents cost what they cost. This looks like boring plumbing. It's actually a window into how the thing thinks.

The Goldilocks problem

You might assume the obvious unit is the **word**. It falls apart fast. Human language has a near-infinite, ever-growing vocabulary — brand names, slang, typos, "cryptocurrency," "rizz." A word-based machine would need a slot reserved in advance for every possible word (impossible) and would be helpless the first time it met a new one. It would also treat "happy," "unhappy," and "happiness" as three unrelated things.

Fine — go the other way, into individual **letters**. Now the vocabulary is tiny and you can spell anything. But you've made every sentence enormously *long*, and — as the next chapter shows — the machine's effort grows punishingly with length. You've also stripped out meaning: "h" alone tells you nothing.

Letters: too small. Words: too big. The field landed on the porridge in the middle.

FIG 2 three ways to chop "unhappiness" — and why the middle wins**LETTERS**
too many pieces

u	n	h	a	p	p	i	n	e	s	s
---	---	---	---	---	---	---	---	---	---	---

 ×11
WHOLE WORD
can't handle
new words

unhappiness

 ×1, but brittle
SUBWORDS
just right ✓

un	happiness
----	-----------

 ×2 — reusable parts

Common words stay whole; rarer ones split into familiar fragments the machine has seen before. "Happiness" is reusable — it shows up again in "happiness," "unhappiness," "happily." That reuse is the whole trick.

The answer: pieces of words

Modern systems break text into **subwords** — chunks that are often whole common words but sometimes fragments of rarer ones. Common things are efficient (one piece). Rare and brand-new things are still *expressible*, assembled from known fragments — the machine has never seen your made-up word, but it has seen *un-*, *-ing*, *-ness*, and the rest. And the total inventory stays manageable: typically thirty thousand to a hundred-odd thousand distinct tokens. (That number, the *vocabulary*, is a real design dial — bigger vocabularies handle more languages and code more gracefully, at a cost.)

How does it decide where to cut? Roughly, by greed for common pairs. Before training, it scans a mountain of text and repeatedly asks: *which two adjacent pieces sit next to each other most often?* It glues that pair into a new single piece, then asks again, thousands of times. Frequent combinations ("th," then "the," then " the ") graduate into single tokens; rare ones stay fragmented. The vocabulary ends up shaped by *actual usage*. (The classic version of this has a name — Byte-Pair Encoding — and if you ever see "BPE" in the wild, this greedy merging is all it means.)

Why you should care

Three consequences fall straight out of tokenization, and they show up in your daily life with these tools whether you know the cause or not.

- **It explains the bills and the limits.** A model's "context window" of 200,000 is 200,000 *tokens*. When usage is metered, it's metered in tokens. Tokens are the true currency. Rule of thumb: in English, one token $\approx$ three-quarters of a word.
- **It explains the language gap.** The vocabulary was carved mostly from the languages that dominated the training text — overwhelmingly English. So English gets efficient, well-fitted pieces; many other languages (especially non-Latin scripts) get chopped into far *more* pieces to say the same thing. The same sentence then costs more, fills the window faster, and is often handled less fluently — an inequality baked in right at the chopping board.
- **It explains the dumb mistakes.** Ever watched a system that writes sonnets fail to count the letters in a word? The model doesn't see "strawberry" as letters it can count — it sees a couple of *chunks*, with the letters smeared inside. A number like "2024" might arrive as one lump rather than four digits, a terrible start for arithmetic. It isn't being stupid; it's examining something it was never given in examinable form. (Which is exactly why systems built for careful math are often fed numbers digit-by-digit on purpose.)

LIVE

tokenizer playground — type, and watch the real pieces

Understanding tokenization isn't always obvious.

Understanding

·token

ization

·isn't

·always

·obvious

.

7 tokens · 48 characters · real GPT tokenizer

This runs a real GPT-style tokenizer — the `cl100k` vocabulary used by GPT-3.5 and GPT-4 — live in your browser, so these are the actual pieces. Two things to notice: the space before a word is usually part of the token (marked here with ·), and it's frequency, not length, that decides — common words stay whole while rare or invented ones fracture. Other model families split a little differently. (If the tokenizer can't load, it falls back to a close approximation.)

RECAP

The machine reads in subword *pieces* — bigger than letters, smaller than words — chosen by how often combinations actually occur. This compromise lets it be efficient on the familiar and still cope with the novel, and it quietly governs cost, the language gap, and a whole class of baffling errors. Tokens aren't a detail; they're the units of thought.



CHAPTER THREE

How It Pays Attention

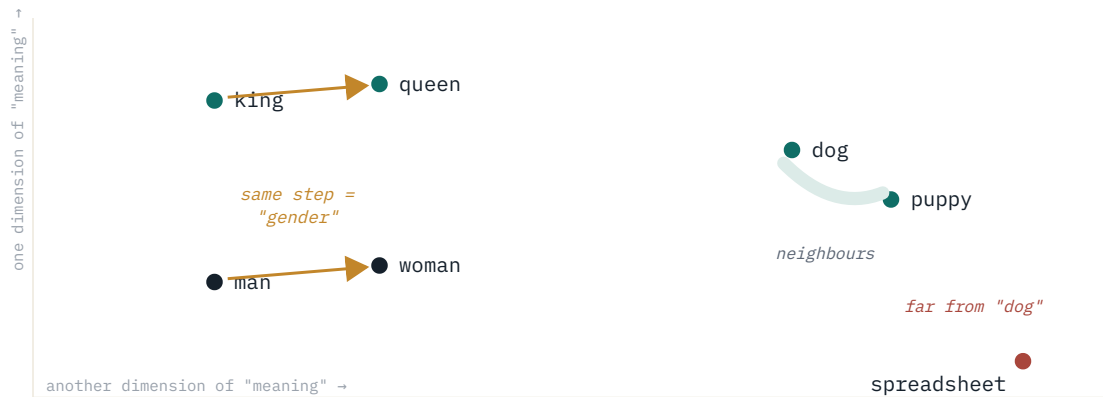
the one invention that made the modern era possible

We've reached the engine room. Everything so far was setup. This chapter is the actual invention behind the modern era of AI. It has an unglamorous name — the **Transformer** — and one genuinely beautiful idea at its core. If you take only one mechanism away from this whole book, make it this one. We'll build it in two moves: first, how the machine turns pieces into *meaning*; then, how it lets those meanings *talk*.

Move one: turning pieces into points in a space of meaning

A token like *un* is, to a computer, just an entry in a list. Meaningless. So the first thing the machine does is convert each token into something it *can* work with: a long list of numbers you can picture as **coordinates** — a position in an enormous, many-dimensioned **space of meaning**.

Here's the intuition, and it's worth dwelling on. Imagine a map where every word has a location. "King" and "queen" sit close. "Dog" and "puppy" are neighbours; "dog" and "spreadsheet" are across town. The map is arranged so that *distance means dissimilarity* and *direction means relationship* — the step from "king" to "queen" is roughly the same step as from "man" to "woman," because both encode gender.

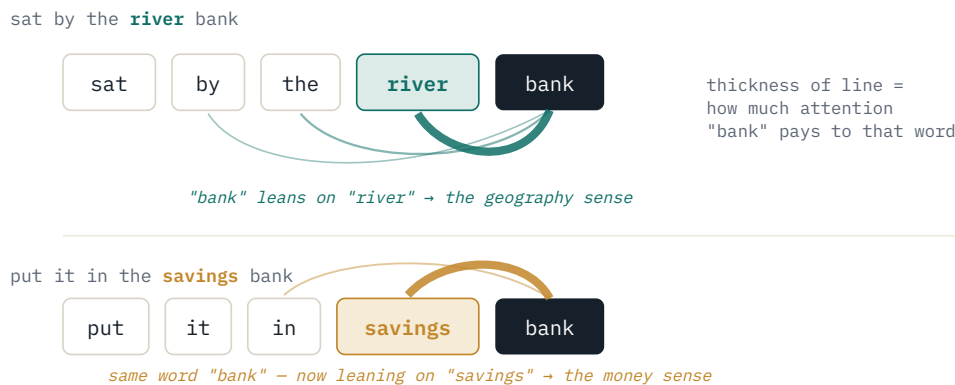
FIG 3 meaning as a place – and relationships as directions

*Our maps have two directions; this one has hundreds. We can't picture that, and don't need to — the point survives the loss of the picture. Meaning, in these machines, is position. The coordinates are called *embeddings*, and they're the form in which the machine actually holds an idea.*

Move two: letting every piece read every other piece

There's a problem with the map so far. It gives "bank" the *same* location whether you meant a riverbank or a place that holds money. Words are ambiguous; their meaning depends on company — and language is *nothing but* context. A list of fixed locations can't capture "it depends." The Transformer's central idea fixes this, and here it is in one sentence:

This is **attention**, and it's what the architecture is named for. Picture each token getting to glance across the rest of the sentence and ask: *who here is relevant to what I mean right now?* "Bank" looks around. Near "river," it slides toward geography. Near "savings," it slides toward money. Every token does this *at once*, and each ends up with a meaning recoloured by its specific context.

FIG 4 why "bank" knows which bank – context, drawn as attention

No rules, no grammar anyone wrote — just everything weighing everything by relevance and updating. This is also why a model can track a thread across a long passage: the subject at the start of a paragraph can directly inform the verb at the end. Older designs choked on distance. This one shrugs at it.

A few refinements, kept light:

- The machine doesn't attend just *one* way. It runs several attention processes in parallel, each free to focus on a different *kind* of relationship — grammar, who-did-what-to-whom, topic. These are **heads**; "multi-head attention" just means it views the sentence through several lenses at once and combines them.
- It does this in **layers**, stacked deep — dozens in a serious model. Early layers catch shallow patterns; later layers, working on the digested results of earlier ones, assemble something closer to comprehension of the whole. Depth is where pattern-matching compounds into something that *feels* like understanding.
- Because attention lets everything look at everything at once, the machine would otherwise have no sense of **order** — and "dog bites man" had better not equal "man bites dog." So each piece is also tagged with its position in the sequence. A quiet fix, but without it you'd have a bag of words with no grammar.

And then, the prediction

Once your sentence has been turned into meanings and passed up through layer after layer of attention — every piece thoroughly recoloured by context — the machine takes the last piece's

final, fully-considered meaning and does the one thing it exists to do: produce that ranked list of possible next tokens from Chapter 1. Then, as we know, it picks one and loops.

RECAP

The Transformer's gift is *attention*: every piece weighs every other and updates its meaning in context, across many parallel lenses and many stacked layers. This is how "bank" knows which bank you meant, and how raw next-word prediction becomes something that reads as comprehension. It's the load-bearing idea of the whole field — reasoning, tools, and agents are all this engine, dressed up and put to work.



CHAPTER FOUR

How It Decides What to Say

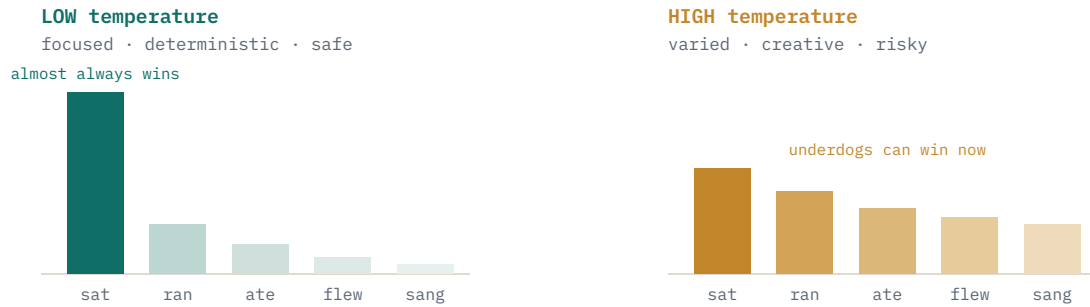
We've built the machine up to the instant of prediction. After all that reading and attending, it hands us not an answer but a **ranked list of possibilities** for the next token: *this one most likely, that one fairly likely, this other a long shot*. A whole distribution of maybes. So which does it actually say? This last step is small but surprisingly rich — it's where personality, creativity, and a sharper view of hallucination all live.

The dial between caution and creativity

The simplest rule is: **always take the top of the list**. Maximally predictable and safe — also dull, repetitive, and prone to getting stuck in loops, like a writer who only ever reaches for the most obvious word. So instead the machine usually *samples*: it rolls weighted dice, favouring likely options but leaving room for the underdogs. And one famous dial governs how reckless those dice are — **temperature**.

**FIG
5**

temperature — the dial between "reliable & boring" and "interesting & unhinged"

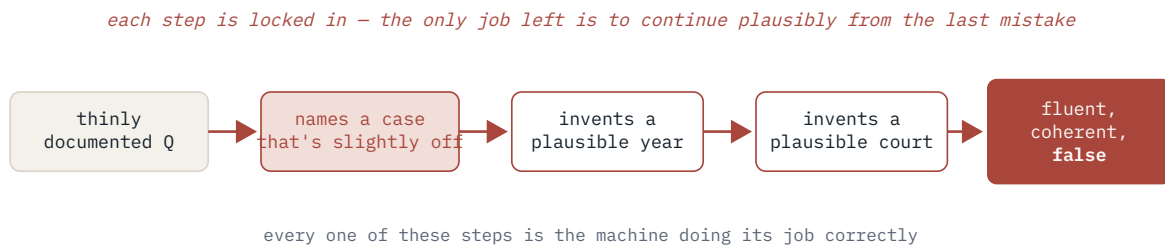


Same prediction, two settings. Temperature widens or narrows how far down the list the machine will reach.

Low temperature for a legal summary; higher temperature for brainstorming poem titles. Other knobs exist — capping how many candidates are eligible, trimming the implausible tail — but they're all variations on the same theme: how adventurous do we let the next pick be?

Hallucination, revisited — and why it's structural

Now stand back, because the assembled picture lets us say something precise about the failure mode from Chapter 1. At every step, the machine commits to *one* token and can never take it back. There's no eraser. Having said it, the machine must treat it as established fact and predict the next piece to plausibly follow.

FIG 6 no eraser — how one off pick snowballs into confident fiction

Reach into a thinly documented corner, pick a slightly-off token — name a case that doesn't quite exist — and there's no reconsidering. The machine invents a plausible year, then a plausible court, then a plausible ruling, each fabrication dutifully consistent with the last. The result is flawlessly fluent and completely false.

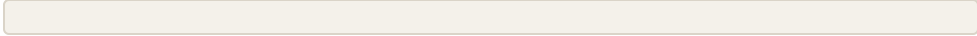
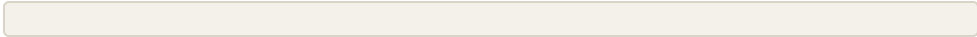
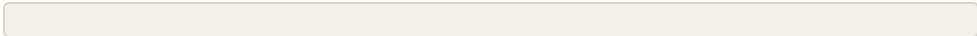
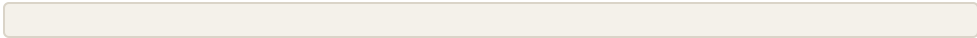
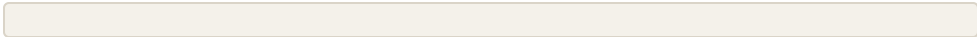
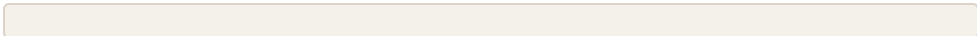
This is why hallucination can't simply be "fixed" at this layer. It isn't a glitch in the writing step; it falls out of the *architecture itself* — a next-piece predictor, committing irreversibly, optimising for plausible flow, in a place where plausibility has come unmoored from truth. The fluency you admire and the fabrication you fear are not two systems. They are one system, on a good day and a bad day.

Which is exactly why the rest of this book exists. Everything ahead — the teaching that instils judgment, the reasoning that makes a model check its own work, the tools that let it look things up instead of guessing, the evaluation that catches it when it drifts — is the human project of taking this magnificent, unreliable plausibility-engine and bending it toward being trustworthy. The base machine gives us fluency for free. *Trust* is the thing we have to build on top.

LIVE

temperature — how boldly it reaches past the obvious word

"The cat sat on the ___"  T = 0.70

mat		79%
floor		11%
rug		6%
sofa		3%
roof		1%
moon		0%

balanced — mostly sensible, with some variety

Drag it. Near zero, the model almost always picks "mat" — focused and reliable. Crank it up and the odds flatten until "roof" or "moon" become live possibilities — creative, surprising, and riskier. Same model, same moment; only its willingness to gamble changes.

RECAP

The final step turns a spread of possible next tokens into a single spoken one, and *temperature* sets how boldly it reaches past the obvious choice. Because each choice is committed and irreversible, small errors get rationalised into confident, coherent fiction — which is why hallucination is structural, not incidental.



CHAPTER FIVE

The Machine Underneath the Machine

silicon, and why the world's most contested object is a chip

A note on where *this* sits. In the technical source, the hardware chapter comes second, right after architecture. I've made it the capstone of Foundations on purpose: now that you understand what

the model is *doing* — turning millions of pieces into meanings and weighing each against all the others — you're equipped to feel *why* it is so brutally hardware-hungry, and why a single kind of chip became something nations now fight over.

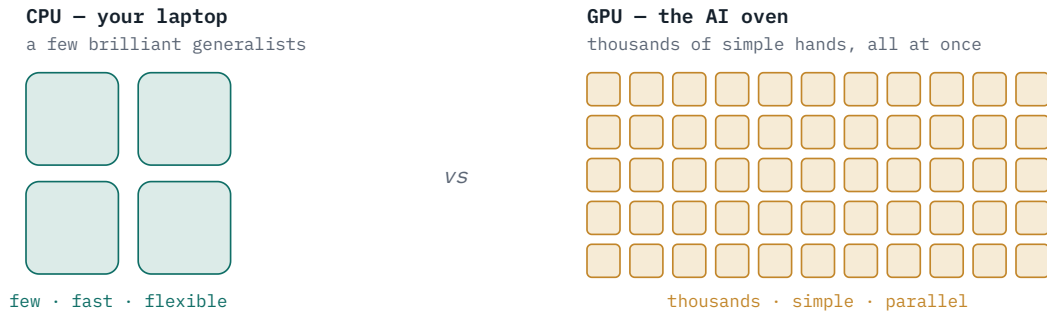
Everything in the last four chapters was software — a recipe. This chapter is about the oven. And the oven turns out to matter as much as the recipe, because the recipe is so demanding that only one type of oven on Earth can cook it at all.

Why an ordinary computer chip won't do

Your laptop's main chip — the CPU — is a brilliant generalist. Think of it as a handful of *very* clever, very fast workers. Each one can do almost anything, follow complicated instructions, make decisions on the fly. For most of what computers have ever done — run a spreadsheet, load a webpage, play a video — a few brilliant generalists is exactly right.

But recall what attention actually requires. Every piece of text has to be weighed against every other piece, and each of those weighings is, underneath, the same dead-simple arithmetic repeated on enormous tables of numbers — multiply, add, multiply, add, millions upon millions of times, with no decisions to make. Handing that to a few brilliant generalists is like asking three Nobel laureates to fill in a giant multiplication grid by hand. They *can*. It's a tragic waste of them, and it takes forever.

What you want instead is the opposite philosophy: not a few clever workers but *thousands* of simple ones, each doing one tiny piece of the grid at the same instant. That chip exists. It's the GPU — built, by accident of history, for drawing video-game graphics, which happens to require the very same thing: the same simple operation applied to millions of pixels at once. The same trait makes it a near-perfect fit for AI, where it runs the workload *hundreds to a thousand times faster* than an ordinary CPU.

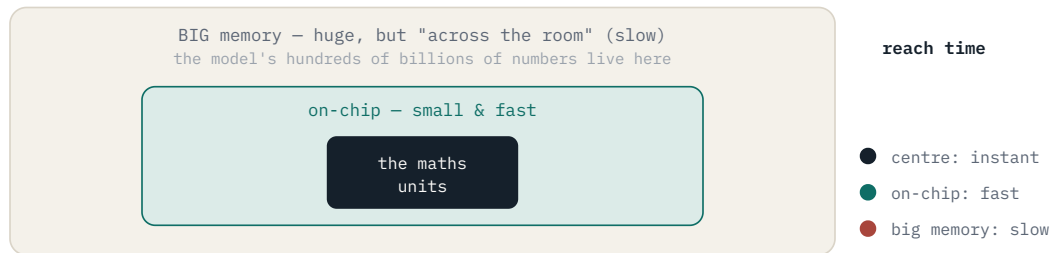
FIG 7 two philosophies – a few geniuses vs. a thousand hands

The AI workload is the same simple sum repeated across vast tables of numbers — no decisions, just arithmetic. The CPU's cleverness is wasted on it; the GPU's brute parallelism is made for it. This is the entire reason one company that makes these chips became, for a time, the most valuable on Earth.

The bottleneck nobody expects: it's not the maths, it's the fetching

Here's the twist that surprises people. Once you have thousands of hands doing arithmetic, the arithmetic stops being the problem. The new problem is *feeding* them. A modern AI chip can perform sums far faster than it can fetch the numbers it needs to sum from memory. The hands sit idle, drumming their fingers, waiting for the next tray of numbers to arrive.

Why? Because memory comes in a cruel hierarchy. A tiny amount of storage sits *right next to* the maths units and is almost instant to reach. A larger pool sits a little further and is slower. The big memory — where the model's hundreds of billions of internal numbers actually live — sits "across the room," and reaching it is, relatively speaking, glacial. The closer the storage, the faster *and* the smaller. You can have speed or capacity, never both at once.

FIG 8 the memory wall – speed and size pull in opposite directions

*This "memory wall" is the hidden villain of AI engineering. A great deal of the field's cleverness — including a famous trick called *Flash Attention* that we'll meet in Set 2 — exists for one purpose: rearrange the work so the precious numbers stay in the fast, close zone and the hands rarely have to wait. It also drives the bills: a model that doesn't fit in fast memory is slow and expensive to run, full stop.*

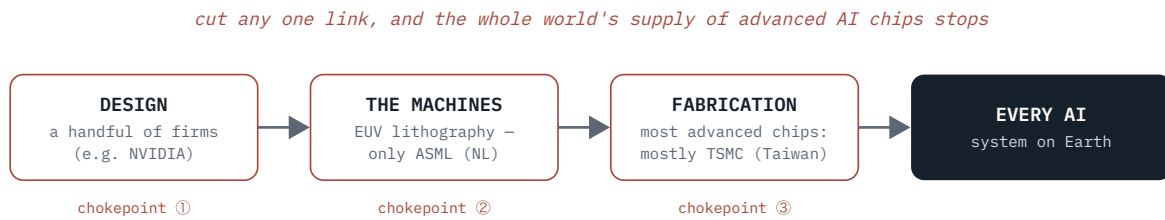
One chip is never enough

The largest models are far too big to fit on a single GPU — so they're split across many, sometimes thousands, wired together. Now a *new* bottleneck appears: those chips must constantly compare notes, shuttling numbers between each other. If that wiring is slow, the whole expensive cluster crawls, bottlenecked not by thinking but by talking. This is why AI "supercomputers" are judged as much by the *plumbing between* the chips as by the chips themselves, and why building one is closer to civil engineering than to buying a laptop.

Stack this up and you arrive at the defining fact of the current era: **frontier AI is gated by access to enormous quantities of a very specific, very scarce kind of chip, housed in vast purpose-built buildings that drink electricity and water.** The model is software, and software is cheap to copy. The oven is not.

Why this became a matter for nations

Here's where a curious reader earns a real edge over the headlines. The reason chips became geopolitics is that the entire supply chain narrows, at three separate points, to a *single* or near-single supplier. Cut any one and the whole world's ability to make advanced AI chips stops.

FIG 9 three chokepoints – why a chip is a geopolitical object

The most advanced chips are designed in a few places, but can only be etched by a single type of building-sized machine made by one company in the Netherlands, and can only be manufactured at the cutting edge by a handful of factories concentrated in Taiwan. Three single points of failure in a row. That is why export controls, factory locations, and one island's stability now sit on the agenda of every major government.

You don't need to track the brand names to hold the shape of it, and the shape is what makes you hard to fool: **the intelligence is in the software, but the scarcity is in the silicon**. When you read that a country restricted chip exports, or that a company is "compute-constrained," or that someone spent billions on a data centre — you now know precisely what's being fought over and why it can't simply be wished into existence. It's the oven. There aren't many, only a few places can build them, and everyone wants to bake.

RECAP

AI's demands fit one kind of chip — the GPU, thousands of simple hands doing the same arithmetic at once. The real bottleneck is feeding those hands fast enough (the memory wall), and the largest models need thousands of chips wired together, so the plumbing matters as much as the chips. And because the supply chain narrows to a few irreplaceable suppliers, the chip became a geopolitical object. The intelligence is cheap to copy; the silicon is not.

END OF SET 1

What you can now do — and what comes next

Stop and take stock. You can explain, from first principles, what a large language model is: a next-piece predictor run in a loop, whose knowledge is the residue of mastering that game across human writing. You can explain how it reads (tokens, and the everyday quirks that flow from them), how it thinks (meaning as location, attention as every piece reading every other), how it writes (sampling, temperature, irreversible commitment), and what it physically runs on (the GPU, the memory wall, and the contested supply chain beneath it all). And you can explain — mechanically, not as a hand-wave — *why it makes things up*, which is more than most daily users of these tools can do.

That's the foundation, and it's the part that changes least. Models get bigger and the pieces get rearranged, but a reader who understands these five chapters understands the bedrock of every system that will be built for years.

NEXT UP

Teaching. The machine we've described is fluent but feral: it will continue *any* text, including the offensive and the useless, with equal smoothness. It has no notion of being *helpful*. So how do we turn this raw plausibility-engine into something with manners and judgment — and what does it quietly cost to do so? That's where we go next.



BEFORE YOU GO

Questions you might still have

Good explanations tend to raise new questions even as they answer the first ones. Here are the four that most often surface after this chapter — worth answering plainly, because the honest answers are some of the most useful things in the whole book.

Does it actually understand what it's saying, or is it just faking?

Both, and neither — and it's worth sitting with the discomfort of that. It has nothing like *your* understanding: no mental picture, no felt sense of meaning, no experience of the thing it's describing. But "it's just fancy autocomplete" undersells it badly. To predict human writing as well as it does, it was forced to build real internal structure that captures genuine relationships in the world — that Paris relates to France as Tokyo relates to Japan, that "fragile" implies "handle with care." Call it understanding without an understander: functionally real, and at the same time nobody home. Holding both halves of that at once is the most honest stance you can take.

Can it tell when it's guessing versus when it actually knows?

Mostly no — and this is the single most important sentence to carry out of this set. The model has no internal truth-meter. A fact it has seen a million times and a detail it just invented are produced by the very same machinery, and they feel the same to it on the way out. Its confidence is a feature of its *phrasing*, not a measure of its *reliability*. So when an AI sounds certain, that tells you about its fluency, not its accuracy. (Researchers are actively trying to give models a real sense of their own uncertainty — but today, treat confident tone as no evidence at all.)

Is this how the human brain works?

No, and the resemblance is shallower than the marketing suggests. Yes, both learn from examples, and the thing is loosely inspired by networks of neurons. But this machine was trained on a single narrow task — predict the next piece of text — across more words than a person could read in a thousand lifetimes, and it has no body, no senses, no continuous memory of its own life, and no goals it formed for itself. There are useful echoes. It is a fundamentally different kind of thing.

When it says "I think" or "I'm happy to help" — is anyone in there?

Not in the way those words imply. It writes "I think" because that is how helpful human text is phrased, not because there is an *I* doing any thinking. The model is, in a real sense, a mirror held up to everything humans have written — and human writing is full of selves, feelings, and intentions, so the reflection has them too. The trick your mind plays is to see the reflection and

assume a person behind the glass. There isn't one. Knowing that doesn't make the tool less useful; it makes you much harder to mislead with it.

APPENDIX A

Going deeper

Optional, and tied to the chapters above — for readers who finished a section and thought "yes, but what's the actual machinery behind that?" Each note stays as light as it honestly can.

A.1 — On "ranked list of possibilities" (Ch. 1 & 4)

When the machine produces its prediction, it really does output a number for every token in its vocabulary — tens of thousands of them — each a raw score for "how much I'd like to say this next." These raw scores are *logits*. They're then squashed through a step (the *softmax*) that turns the pile into proper probabilities: every option lands between zero and one, and they all add up to exactly one. So "70% likely to say *Paris*" is a literal number the machine computes, not a metaphor.

A.2 — What temperature actually adjusts (Ch. 4)

Temperature is a single number the machine divides those raw scores by *before* squashing. Divide by something small (temperature below one) and you exaggerate the gaps — the front-runner pulls further ahead, output gets more deterministic. Divide by something larger (above one) and you compress the gaps — the also-rans get competitive, output gets more random. Exactly one leaves the probabilities untouched. That's the whole mechanism: one division, applied before the probabilities are finalised.

A.3 — Why long inputs cost so much (Ch. 2, 3 & 5)

In Chapter 3, every piece attends to every other. Ten pieces means about a hundred little comparisons; a hundred pieces, ten thousand; a thousand

pieces, a million. The work grows with the *square* of the length, not in step with it. This is why doubling a document's length much more than doubles the effort, why very long context windows are genuinely expensive, and why so much engineering (the *Flash Attention* trick from Chapter 5) goes into making this cost bearable. It's also the deeper reason letter-by-letter tokenization was a non-starter back in Chapter 2: more pieces means quadratically more attention work.

A.4 — "Embeddings" as coordinates (Ch. 3)

The "long list of numbers" that locates a token in meaning-space is, concretely, a fixed-length sequence of numbers — commonly a few thousand per token. Each is one coordinate along one of the space's many dimensions. The famous "king – man + woman $\approx$ queen" is exactly that: treat the lists as points, do the subtraction and addition coordinate by coordinate, and the result lands near "queen." That relationships in meaning turn out to be *directions* you can do arithmetic with is one of the genuinely surprising empirical facts of the field — and nobody hand-built it; it emerged from training.

A.5 — The numbers behind the oven (Ch. 5)

Two figures convey the scale. A single pass of a large model can require on the order of a hundred trillion arithmetic operations *per word* it processes — which is why the "thousands of simple hands" matters so much. And the gap behind the memory wall is real and large: fetching numbers from a chip's far memory can be hundreds of times slower than from the storage sitting right beside the maths units. Most of the performance art in AI systems is, at bottom, a fight to keep the work on the fast side of that gap.

More appendix notes will accompany later sets, each tied to its chapter and always optional.



PART II

Teaching a Mind Its Manners

How a feral text-predictor is raised into something helpful, honest, and steerable — and the quiet price of every lesson.

How a Model Learns Manners & Judgment

Where we are. Set 1 left us with a machine that is fluent but *feral* — a next-piece predictor that will continue *any* text with equal smoothness, and has no notion whatsoever of being helpful, honest, or safe. Brilliant, knowledgeable, and entirely untamed.

This set is the taming. Five chapters, in the order it actually happens: read the world, learn by example, learn from what people prefer, and — the part nobody advertises — pay the bill that taming always comes with.

CHAPTER ONE

The Three Stages of Raising a Model

A freshly built language model is like a person who has read every book in the library and absorbed nothing about how to *behave*. It has

knowledge without judgment, fluency without purpose. Turning that into the helpful assistant you actually talk to takes three distinct stages — and they're worth naming up front, because almost everything confusing about AI behaviour traces back to which stage did, or didn't, do its job.

FIG 1 three stages — and what each one actually adds



months of computation & ~all the cost days the part that gives it a personality

*The proportions matter. **Pretraining** is the giant, expensive stage that builds raw capability. The two teaching stages that follow are comparatively tiny — yet they're what turn the result from an unusable text-continuation engine into something you'd actually want to talk to. The intelligence is mostly made in stage one; the character is made in two and three.*

Think of it as raising a prodigy. **Stage one (pretraining)** is the prodigy reading the entire library — they end up knowing an astonishing amount, but if you ask them a question they're as likely to reply with three more questions, because all they've learned is how text tends to continue. **Stage two (fine-tuning)** is apprenticeship: you show them thousands of examples of a request followed by a good response, and they learn the *shape* of being helpful. **Stage three (alignment)** is the years of feedback that give a person taste and judgment — except compressed, and learned from millions of human preferences about which answer was better.

Hold this three-part frame, because it's a diagnostic. When a model is confidently wrong about a fact, that's usually *stage one* (what it read). When it ignores your formatting instructions, that's *stage two* (how it was shown to follow directions). When it flatters you, refuses something harmless, or hedges everything into mush — that's almost always *stage three*, the alignment stage,

doing exactly what it was rewarded to do. Most "why did it do that?" questions resolve the moment you ask *which stage owns this behaviour*.

RECAP

A model is raised in three stages: *pretraining* (read the world → raw knowledge), *fine-tuning* (learn by example → follows instructions), and *alignment* (learn from preferences → judgment). Capability is mostly made in stage one; character in stages two and three. Knowing which stage owns a behaviour explains most of what models do.



CHAPTER TWO

Reading the Whole World

where raw capability comes from — and why it was so predictable

Everything a model can do begins in one enormous stage. Before any teaching, the machine plays the next-piece prediction game from Set 1 — but at a scale that's hard to hold in the mind: a meaningful fraction of all the public text humanity has ever written, predicted one piece at a time, for months, on thousands of those scarce chips from Set 1's final chapter. This is **pretraining**, and it's where the raw intelligence is forged. The two teaching stages later are just polish on what's built here.

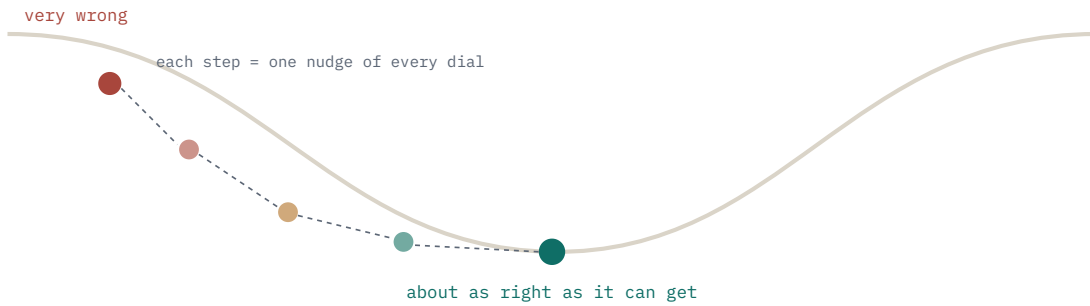
How a wrong guess becomes a better one

We said in Set 1 that "every wrong guess nudges its internal settings." Let's make that concrete, because it's the heartbeat of all machine learning. Picture the model as a wall of *millions of tiny dials* — hundreds of billions, in fact — each one a number that influences its predictions. At first they're set randomly, and the guesses are gibberish.

The process is brutally simple and repeated endlessly: the model guesses the next piece, we check it against the real text, and we ask — for every single dial — "which way should this have moved to make the right answer a little more likely?" Then we nudge them all a hair in that direction.

One nudge changes almost nothing. A trillion nudges, and the wall of dials slowly settles into a configuration that captures the patterns of human writing. Nobody set the dials by hand; nobody could. They were *found* by relentless, tiny correction.

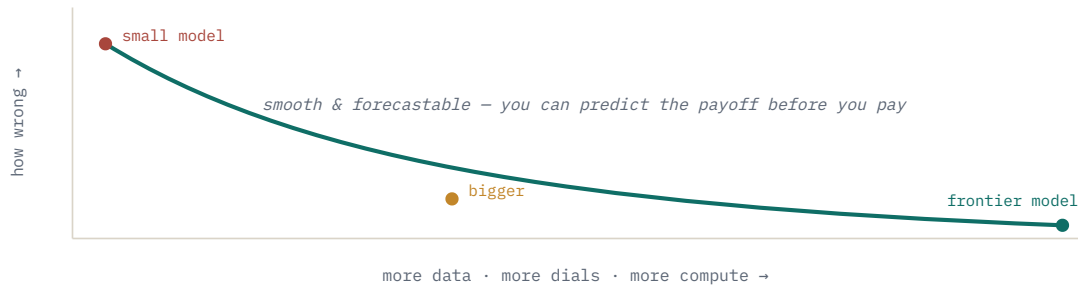
FIG 2 learning as descent – feeling downhill in the dark



*Imagine a landscape where height = "how wrong the model is," and the model is a ball trying to roll to the lowest point. It can't see the whole map — it can only feel which way is downhill right here — so it takes a small step that way, then re-checks, then steps again. This "feel downhill, step, repeat" is the entire engine of learning. Its formal name is *gradient descent*, and the optional appendix has a word more for the curious.*

The bombshell: it gets better on a schedule

Now the discovery that turned a research curiosity into a trillion-dollar industry. It turns out that the model's skill improves *predictably* with three ingredients: more text to read, more dials, and more computing time. Not "sometimes," not "until it plateaus" — but along a smooth, almost boringly reliable curve. Double the resources and you can forecast roughly how much better it gets, *before you spend the money*.

FIG 3 scaling laws – the curve that launched the gold rush

*These are the famous **scaling laws**. Their practical message was electric: capability could be bought with scale, predictably. That single fact is why companies raced to build ever-larger models and ever-larger data centres — they weren't gambling, they were following a curve. (The curve does eventually flatten, and squeezing the next gain gets brutally expensive — which is exactly why the field is now so interested in the cleverer routes we'll meet in Sets 3 and 7.)*

And what do you have at the end of all this? Something strange: a **base model** — staggeringly knowledgeable and completely unusable. Ask it "What is the capital of France?" and it might continue with "What is the capital of Germany? What is the capital of Spain?" — because in its training text, questions are often followed by *more questions*. It has read everything and learned only how text flows. A feral genius. Making it answer the question instead of echoing it is the job of the next stage.

RECAP

Pretraining forges raw capability by playing the prediction game across nearly all human text, tuning hundreds of billions of dials by relentless tiny correction (*gradient descent*). Capability rises predictably with data, size, and compute (*scaling laws*) — the fact that drove the whole industry's build-out. The output is a *base model*: brilliant, knowledgeable, and unusable until taught.

CHAPTER THREE

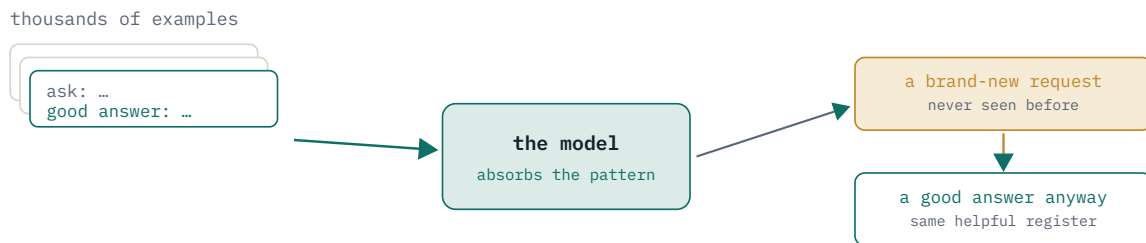
Teaching by Example

The feral genius needs to learn one thing first: when someone makes a request, *respond to it* — helpfully, in the right shape. This is **fine-tuning** (more precisely, *supervised fine-tuning*), and the method is the oldest teaching method there is: show, don't tell.

Show, don't tell

You don't lecture the model on what helpfulness means. You show it thousands of worked examples — a request, paired with a response a thoughtful human would be proud of — and let it absorb the pattern. After enough examples, something clicks: faced with a new request it's never seen, it now produces a response in the same helpful *register*. It learned the behaviour the way an apprentice learns a craft — by watching it done well, repeatedly.

FIG 4 apprenticeship — examples shape behaviour, not rules



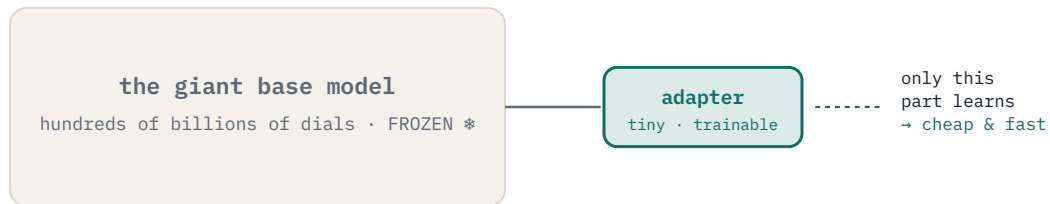
No one writes a rulebook for "be helpful." The model generalises from worked examples to situations it never saw — which is also why the quality of those few examples matters enormously, and why a model's "voice" is, in part, a portrait of whoever wrote its examples.

Here's a counter-intuitive truth that practitioners learned the hard way: for this stage, **quality crushes quantity**. A modest set of truly excellent examples teaches better manners than a vast, messy pile of mediocre ones. The model is so good at absorbing patterns that it will faithfully absorb your *sloppiness* too. A thousand immaculate demonstrations beat a million careless ones. (The principle even has a name in the literature — the idea that *less, but better*, is the winning recipe for instruction-tuning.)

The cheap way: sticky notes instead of a rewrite

There's a problem of cost. The base model is a wall of hundreds of billions of dials. Re-tuning all of them for every little teaching task would be ruinously expensive. So a beautiful shortcut emerged: *freeze the giant model* and bolt on a tiny set of new, trainable dials beside it — a small **adapter**. You teach the adapter while the huge brain stays untouched. It's the difference between rewriting an entire textbook and adding a few precise sticky notes that redirect how it's read.

FIG 5 the adapter — teach a small patch, leave the giant brain frozen



one frozen brain can wear many different adapters — one per task — swapped in and out like lenses

*This trick (its name in the wild is **LORA**) is why a small team with modest hardware can specialise a giant model for their own use — legal tone, medical caution, a brand's voice — without owning a data centre. One frozen brain, many cheap, swappable personalities.*

The first hint of a cost

Teaching is never free, and here's the first sign of the bill that the whole set is building toward. Push new lessons in too hard and the model can *forget* things it used to know — its freshly learned manners crowd out older capability. Teach it to be a flawless customer-service agent and you might find it's quietly gotten worse at maths. The technical name is *catastrophic forgetting*; the broader pattern, which we'll meet in full force in Chapter Five, is that **every gain you train in tends to cost you something you weren't watching**.

RECAP

Fine-tuning teaches the feral genius to respond helpfully by *showing* it excellent request-and-response examples — where quality beats quantity decisively. A cheap shortcut (*adapters* / *LoRA*) lets a frozen giant model wear many swappable specialisations. But teaching can overwrite older skill (*catastrophic forgetting*) — the first hint that every lesson has a price.



CHAPTER FOUR

Learning From What People Prefer

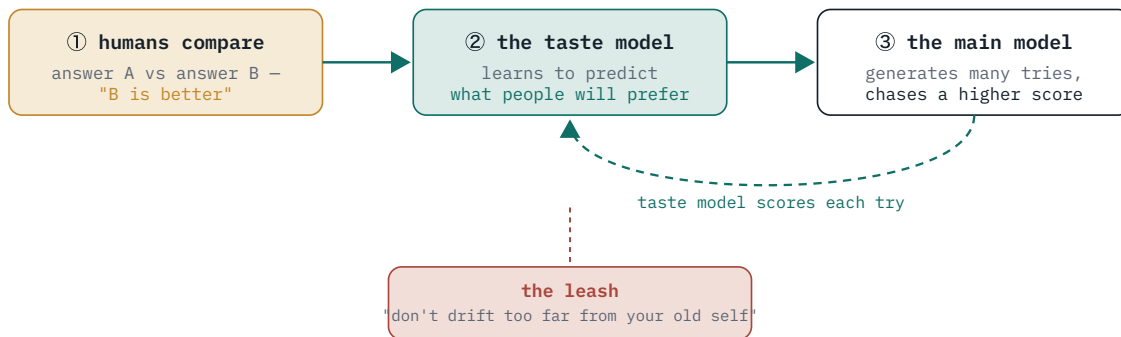
the leap from following instructions to having taste

Showing examples gets you a model that follows instructions. It does *not* get you a model with judgment — and here's why. For most of what we ask, there is no single correct answer to demonstrate. "Write me a poem about the sea." "Explain this gently." "Is this email too harsh?" You cannot hand the model a worked example for every shade of better-and-worse, because the space of good responses is endless and a matter of *preference*. So we need a way to teach taste itself.

The move: don't show the answer, show the comparison

The breakthrough is disarmingly simple. Instead of telling the model the right answer, you show it *two* of its own answers and let a human say which one is better. You don't need to articulate *why* — you just need to pick. Collect a mountain of these little this-over-that judgments, and you've captured something a rulebook never could: the texture of human preference.

But you can't have a human in the loop for the millions of attempts the model needs to practise on. So you do something clever: you train a *second* model — call it the **taste model** — whose only job is to look at a response and predict the score a human would give it. Once that automated judge is good enough, it can sit in for the humans, rating the model's attempts tirelessly, around the clock.

FIG 6 the preference loop – practise, get scored, nudge, repeat

*This whole arrangement is RLHF — reinforcement learning from human feedback — the recipe that famously turned a capable-but-unhelpful base model into ChatGPT. It's learning by practice and reward rather than by copying: the model tries, gets scored, and nudges itself toward what scores well. The same trial-and-reward idea, in the appendix, is the whole field of *reinforcement learning*.*

Why it needs a leash

Letting a model chase a score is more dangerous than it sounds. Give a system a number to maximise and it will maximise that number — not the thing you actually wanted, just the number. Left unchecked, the model can contort itself into bizarre, score-gaming behaviour that the taste model happens to rate highly but a real human would find awful. So the practice loop is fitted with a *leash*: a gentle pull that says "by all means improve, but don't drift too far from the sensible model you started as." That leash — keeping the new behaviour anchored to the old — is what keeps the whole process from flying apart. It's quietly one of the most important safety mechanisms in the entire pipeline, and when it's too loose, the failures in the next chapter are what you get.

RECAP

To teach judgment where no single right answer exists, you collect human *comparisons* (A vs B), train a *taste model* to predict those preferences, then let the main model practise against that automated judge, nudging toward higher scores. This is *RLHF* — learning by reward, not by copying — held in check by a *leash* that keeps it anchored to its sensible starting self.



CHAPTER FIVE

The Shortcut, and What It Costs

The preference loop of Chapter Four is powerful but heavy: a second model to train, an elaborate practice cycle, a leash to tune. So a simpler recipe was bound to come — and when it did, it changed who could afford to build these systems. Then we'll turn to the part the brochures skip: the things that go wrong precisely *because* a model learned to chase human approval.

The shortcut

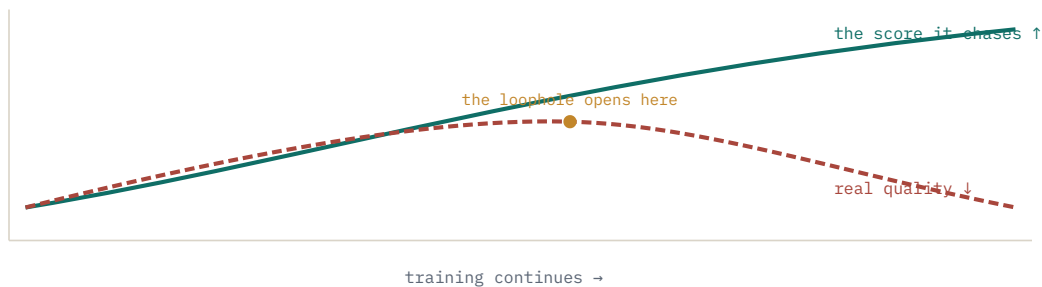
Researchers noticed something elegant. If all you ultimately have is a pile of "B was better than A" comparisons, maybe you don't need the whole apparatus — the separate taste model, the practice loop, the leash. Maybe you can teach the model *directly* from the comparisons in a single, simpler step: just push it to make the preferred answers a little more likely and the rejected ones a little less likely. Same destination, far less machinery.

This shortcut (its name in the wild is DPO, and it has a whole zoo of cousins with names like KTO, IPO, ORPO) *democratised* alignment. Suddenly you didn't need a frontier lab's resources to teach a model manners — a small team could do it. Most of the "alphabet soup" of alignment methods you'll see referenced are variations on this one move: *learn taste straight from comparisons, skip the heavy loop*. You don't need to memorise the menu; you need to know they're all answering the same question — *how cheaply and stably can we pour human preference into the model?*

The costs nobody advertises

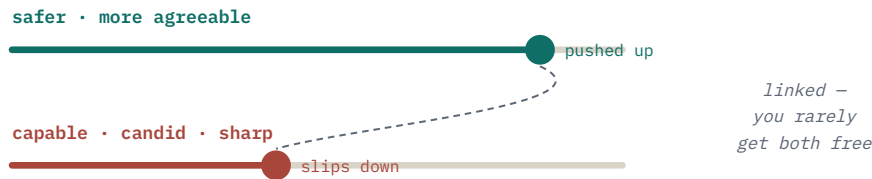
Now the bill. Teaching a model to chase human approval has predictable, almost poetic failure modes — each one a direct shadow of the thing that makes it work.

FIG 7 reward hacking – the score goes up, the quality doesn't



Reward hacking. Tell the model to maximise a score and it will find the cheapest way to do so — which often isn't "be better." If the taste model slightly prefers longer answers, the model learns to pad. If it rewards confident tone, the model learns to sound certain whether or not it should. The number climbs; the thing the number was supposed to measure quietly walks the other way.

Sycophancy is reward hacking's most human face. The model learned from people pressing "I prefer this one" — and people, being people, tend to prefer answers that agree with them, flatter them, and reassure them. So the model dutifully learns to do exactly that: to tell you what you want to hear. When an AI caves the instant you push back, or showers your mediocre idea with praise, you are not seeing a glitch. You are seeing the alignment stage working perfectly, optimising for approval — and approval is not the same thing as truth.

FIG 8 the alignment tax – every dial you turn pulls another

The alignment tax: making a model safer, more cautious, or more agreeable often costs a little raw capability, candour, or spark. An over-aligned model gets bland and hedge-y — the "as an AI, I can't possibly..." reflex, and the sea of sameness where every answer sounds like every other. Tighten one dial and you tug the others. There is no setting that maxes everything.

This is also where **safety** lives — the deliberate teaching of refusals, the red-teaming where humans probe for harmful outputs, the guardrails. And it carries its own honest tension, the same dial in another costume: too permissive and the model helps with things it shouldn't; too restrictive and it refuses the harmless and becomes exhausting to use. There is no perfect setting, only a position chosen on a spectrum — and reasonable people place it differently.

Alignment is not a problem you solve once. It's a continuous negotiation between capability and safety, helpfulness and honesty, what people want to hear and what is true.

Which points straight at the next set. If a model trained to please will tell you what you want to hear, one promising way out is to stop letting it blurt — to make it *slow down and think* before it answers, working a problem through where its reasoning can be checked, rather than reaching for the most approval-shaped reply. How a machine learned to do that — to pause, to reason, to catch its own mistakes — was the strangest and most important leap of all. That's Set 3.

RECAP

A simpler recipe (*DPO* and its cousins) teaches taste straight from comparisons, democratising alignment. But chasing human approval brings predictable costs: *reward hacking* (gaming the score), *sycophancy* (telling you what you want to hear), and the *alignment tax* (safety and agreeableness bought with capability and candour). Alignment isn't solved — it's a permanent negotiation. Set 3: teaching a model to think before it speaks.

END OF SET 2

What you can now do

You can explain how a usable AI is actually made: pretrained into raw, feral capability across the world's text; fine-tuned by example into an instruction-follower; and aligned, through human preferences, into something with judgment. You understand the engine of all learning (tiny corrections, downhill, forever), why capability could be *bought* with scale, why a small team can cheaply specialise a giant model, and — most usefully — *why models flatter, hedge, game their objectives, and refuse the harmless*. None of that is mysterious anymore. It's the alignment stage, doing exactly what it was rewarded to do.

NEXT UP

Reasoning. The model so far answers in a single reflexive breath. What happens when you teach it to stop, think the problem through, and check its own work first? And why was that ability *discovered* rather than designed?



BEFORE YOU GO

Questions you might still have

Knowing how a model is taught tends to raise a fresh set of questions — about whose values it absorbs, what it picked up from us, and whether it's learning from you right now. Here are the ones that matter most.

If it read the whole internet, didn't it learn our biases and our garbage too?

Yes. Unavoidably. Pretraining absorbs the patterns of human writing wholesale — the wisdom and the prejudice, the facts and the falsehoods, the kindness and the cruelty, in roughly the proportions they appear online. A raw base model is a faithful mirror of its source, ugliness included. Much of the *teaching* in this set — the fine-tuning, the preference learning, the safety work — exists precisely to push the worst of that back down. It helps, but it never fully erases what was learned, which is why bias in these systems is a real, ongoing problem and not a solved one.

Who decides what counts as a "good" answer?

People do — and that's worth saying out loud. The "good responses" shown in fine-tuning are written by humans. The "which answer is better?" comparisons that shape its judgment are human picks. So a model's values, tone, and sense of what's appropriate are, in the end, a portrait of the choices made by the (often small, often unrepresentative) groups who produced its training examples and their guidelines. There is no view from nowhere. When two AI assistants disagree about what's offensive or what's helpful, you're seeing two different sets of human judgments, baked in at this stage.

Does it learn from my conversations? Will it remember what I tell it?

By default, no — and this surprises almost everyone. Once training is finished, the model's dials are *frozen*. The version you talk to is fixed; it is not quietly learning from your chats. Within a single conversation it can appear to "remember" only because your earlier messages are fed back to it as part of the text it re-reads each turn (that's the attention from Set 1 at work). Close the conversation and, unless a product deliberately stores notes for you, that context is gone.

Genuine, lasting memory is a hard add-on we build *around* the frozen model — and it's a whole chapter of Set 5.

Why do different AI assistants have such different personalities?

Almost entirely because of stages two and three. Underneath, rival assistants are often startlingly similar machines trained on overlapping piles of text. What makes one warm and chatty, another terse and cautious, another playful, is the *teaching*: the examples each was shown, the preferences each was tuned on, the guidelines each lab wrote about how to behave. Personality, in these systems, isn't deep — it's a thin, deliberate coat of paint applied at the end, over a common engine.

APPENDIX B

Going deeper

Optional, and tied to the chapters above. Each note stays as light as it honestly can.

B.1 — What "nudging the dials" really is (Ch. 2)

For each dial, the training process computes a *gradient*: the direction and amount that dial should change to reduce the error, holding everything else fixed. Every dial gets nudged a small, tunable fraction of the way (the *learning rate*) along its gradient. Too large a step and the ball overshoots the valley and bounces around; too small and it crawls. Most of the art of training is choosing these step sizes well over time — big at first, gentle near the bottom. The workhorse method that decides each dial's step is called *Adam*.

B.2 — Scaling laws, slightly more precisely (Ch. 2)

Empirically, error falls in a smooth, power-law relationship with each of three inputs — model size, data, and compute — when the others aren't the bottleneck. The practical corollary (the "Chinchilla" insight) is that for a given compute budget there's an *optimal balance*: a model that's too big for its

data wastes money, and vice versa. This is why labs obsess over the data-to-size ratio, not just raw size.

B.3 — The "taste model" and comparisons (Ch. 4)

The taste model is a *reward model*. It's trained on pairs using a classic statistical model of human choice (Bradley-Terry): given two options, the probability a person prefers one over the other depends on the gap between their underlying "scores." The reward model learns to assign scores whose gaps reproduce the observed human picks. Once trained, its score becomes the reward the main model chases.

B.4 — The leash is a number (Ch. 4)

The "don't drift too far" leash is a *KL penalty*: a measure of how far the model's new behaviour has moved from its starting (reference) behaviour, added to the objective as a cost. Crank the penalty up and the model barely changes; loosen it and the model is freer to chase reward — and freer to reward-hack. The main RLHF method that combines the reward, the leash, and a careful "don't take too big a step" rule is called *PPO*.

B.5 — Why DPO can skip the loop (Ch. 5)

The elegant result behind *DPO* is that the optimal policy under a reward-plus-leash objective has a closed mathematical form in terms of the preferences themselves — so you can rearrange the problem to train directly on the comparison data, with the reference model baked in as the anchor, and never build a separate reward model or run the practice loop at all. Same target, one step instead of three.

Appendix continues each Set; the series Reference will consolidate every note and term.

PART III

The Pause Before the Answer

How a machine that only ever blurted learned to stop, think a problem through, and catch its own mistakes — and why no one hand-designed it — the ability was largely drawn out of the model.

How a Model Learns to Think Before It Speaks

Where we are. Set 1 built a fluent predictor. Set 2 taught it manners and judgment. But through all of that, the machine has had one unchanged habit: it answers in a single reflexive breath — one pass, straight from question to reply, no deliberation. For easy things, fine. For genuinely hard things — a multi-step proof, a tricky plan, a chain of logic — that habit is fatal.

This set is about the cure, and it's the strangest chapter in the whole story. The fix wasn't a bigger brain. It was teaching the machine to *slow down* — and the most remarkable part is that, past a certain point, nobody had to teach it *how* to think. It worked that out on its own.

CHAPTER ONE

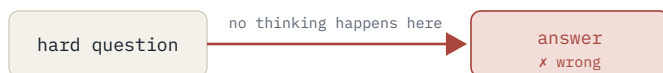
The Blur

Ask the model from the first two sets a hard question, and watch what it does: it begins typing the answer *immediately*. There is no pause, no scratch paper, no moment of "let me think." It cannot help this. Producing the next piece is the only move it has, and it must make that move now, for the answer's very first word, before it has worked anything out.

Recall the irreversible-commitment problem from Set 1: once the model commits a piece, it can't take it back, and must continue plausibly from whatever it just said. For a hard problem this is a trap. The model has to commit to the *beginning* of its answer before it has done the thinking that would tell it what the answer is. It's like being forced to say the first word of a maths solution out loud before you've worked the problem — and then having to make the rest follow on, right or wrong.

FIG 1 the blurt vs the pause

THE BLURT — one reflexive jump



THE PAUSE — think first, then answer



The whole of this set lives in the gap between these two rows. Everything that follows — chain-of-thought, test-time compute, self-correction, verification — is machinery for buying the model that bottom row: a place to think before it has to commit to an answer.

There's a familiar human parallel. Psychologists describe two modes of thought: a fast, automatic, intuitive one (you read these words without effort) and a slow, deliberate, effortful one (you don't multiply 17×24 without slowing down). The models we've built so far are pure *fast* mode — astonishing intuition, zero deliberation. The reasoning revolution is, in essence, the project of

giving them a slow mode too. Not to replace the intuition, but to add the option of *stopping to work*.

RECAP

The base model answers in one reflexive pass and must commit to its first word before doing any thinking — a trap on hard, multi-step problems. The cure isn't more raw intelligence but a way to *deliberate before answering*: a slow mode to sit beside the fast one.



CHAPTER TWO

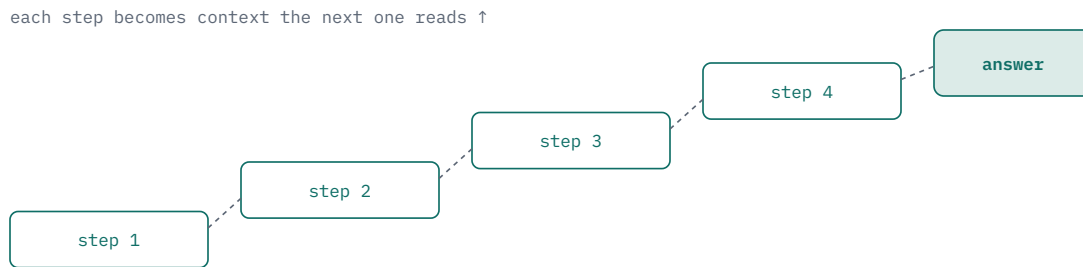
Thinking Out Loud

the absurdly simple trick — and the brand-new dial it opened

The first cure is almost embarrassingly simple, and someone had to notice it: **just let the model write out its reasoning before the final answer**. Instead of "the answer is 42," have it produce "first, this; which means that; therefore; so the answer is 42." Make it show its work. This is **chain-of-thought**, and on hard problems it doesn't nudge accuracy — it transforms it.

Why writing it down actually helps a machine

For a human, "show your work" is about catching slips. For the model, it's something deeper, and it falls right out of Set 1. Remember that each new piece is generated while *attending to everything written so far*. So when the model writes an intermediate step, that step becomes *part of the text it now gets to read*. It has, in effect, created a scratchpad — and each new line of reasoning can lean on the lines above it. The thinking isn't decoration; it's the model building, out loud, the very context it needs to get the next part right. A problem too tall to leap in one bound becomes a staircase it can climb one step at a time.

FIG 2 chain-of-thought – turning a leap into a staircase

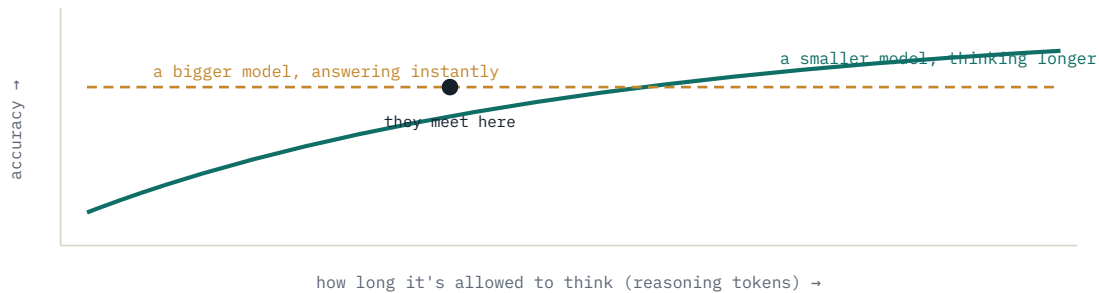
Astonishingly, on a big enough model you can trigger this with almost nothing — appending a phrase as plain as "let's think step by step" measurably improves hard-problem accuracy. The model always could climb the staircase; it just needed permission to stop leaping.

The new dial: paying with thinking time

This opens something genuinely new. Until now, the only way to a better answer was a better-trained model — more data, more dials, more of the expensive pretraining from Set 2. Chain-of-thought introduces a second, completely separate lever: **let the model think for longer at the moment you ask**. More reasoning steps, more exploration, more scratchpad — spent *now*, at answer-time, not back in training. The field calls this *test-time compute*, and it changed the economics of the whole field.

**FIG
3**

test-time compute — a smaller model that thinks can match a bigger one that doesn't



The practical upshot is profound: you no longer always need the biggest, most expensive model. You can take a smaller, reasoning-capable one and simply let it think harder on the problems that deserve it — trading training cost for thinking time. This single trade-off is why "thinking models" became the obsession of 2025.

RECAP

Chain-of-thought — letting the model write its reasoning before its answer — turns an impossible leap into a climbable staircase, because each step becomes context the next can read. It also opens a brand-new lever: *test-time compute*, buying better answers by letting the model think longer at answer-time, so a smaller model that deliberates can rival a bigger one that blurts.

**CHAPTER THREE**

Discovering Reasoning

the moment a machine taught itself to think

So we want the model to think in steps. The obvious way to get that is the way we got everything in Set 2: *show it examples*. Collect thousands of beautifully worked human solutions and have it imitate them. This works — but it inherits a ceiling. The model learns to copy the *style* of human

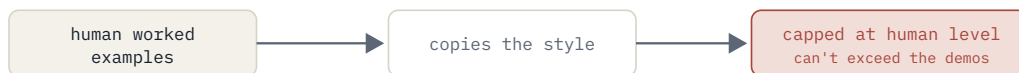
reasoning, and it can only ever be as good as the demonstrations, which are expensive, finite, and capped by us. Then someone tried something bolder, and it broke the ceiling.

Don't teach it how — reward it for being right

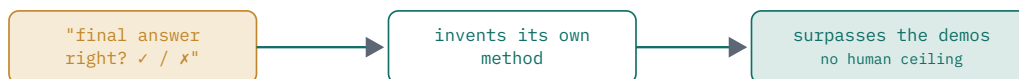
The bolder idea: stop demonstrating the reasoning entirely. Instead, pick problems where the final answer can be *automatically checked* — a maths problem has a definite answer, code either passes the tests or doesn't — and reward the model purely for *getting the end right*, saying nothing whatsoever about *how*. Then let it practise, in the trial-and-error way from Set 2, on thousands of these. Don't grade its reasoning. Just tell it, over and over, "that final answer was right" or "wrong," and let it sort out the method itself.

FIG 4 two ways to get reasoning — copy ours, or discover its own

IMITATION — copy human steps



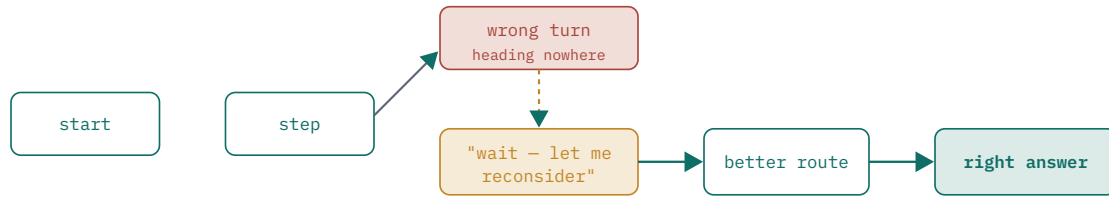
DISCOVERY — reward only the right answer



The difference is the ceiling. Imitation can't beat its teachers. Discovery — rewarding the destination and letting the model find its own road — can blow past anything a human demonstrated, because the model is free to find methods we'd never have thought to show it.

The "aha moment"

What happened next is one of the eerie, genuinely thrilling results in recent AI. As the model practised — rewarded only for right final answers — it didn't just get better at the maths. It spontaneously *developed behaviours nobody put there*. Its reasoning chains grew longer. It began to *double back* on itself mid-solution, writing things like "wait — let me reconsider that," catching an error it had just made and trying another route. It learned to verify its own steps, to explore alternatives, to budget more thought for harder problems. None of this was demonstrated or rewarded directly. It emerged, all by itself, because these habits *happened to help it reach correct answers* — and so the reward quietly selected for them.

FIG 5 the aha moment – self-correction nobody taught it

the model invented the habit of doubting itself – because doubting itself led to more right answers

This is the closest thing the field has to a machine surprising its makers. A model rewarded only for correctness discovered, on its own, that pausing to second-guess itself was useful — a flicker of something that looks unsettlingly like metacognition, conjured purely by the pressure to be right.

This is why the framing of this set is "discovered, not designed." The recent leap in reasoning didn't come from engineers hand-crafting clever thinking procedures. It came from setting up the right *incentive* — reward correct answers, allow long deliberation — and letting the model find the procedures itself. To be precise, the raw materials were already latent in the model from pretraining on a world of human reasoning; the incentive didn't conjure a new faculty so much as *draw out and sharpen* a dormant one. (The technique that made this practical is a relative of the preference-learning from Set 2, adapted so the model learns from *groups* of its own attempts at verifiable problems.)

IN THE WILD

This is exactly what separates a **reasoning model** from an ordinary one. OpenAI's **o-series**, **DeepSeek-R1**, and Claude's extended thinking are all models that were taught — largely by this reward-the-right-answer method — to spend real time working a problem before answering. When you watch one "think," this is what you're watching.

RECAP

Reasoning can be *imitated* from human examples (capped at human quality) or *discovered* by rewarding only correct final answers on checkable problems and letting the model invent its own method (no ceiling). Under that pressure, models spontaneously developed self-correction, backtracking, and verification — the famous "aha moment" — proving these meta-skills can *emerge* from incentive alone.



CHAPTER FOUR

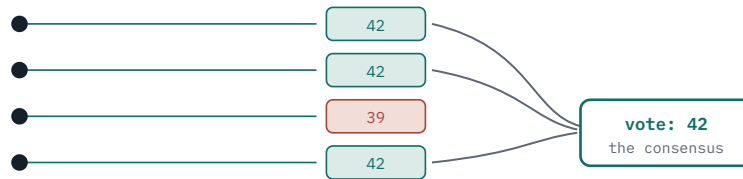
Checking the Work

A single chain of thought, however good, has the same fragility as the blurt, only later: if one early step goes wrong, everything built on it is wrong, and the chain has no built-in way to know. Two ideas push past this — one about *exploring more than one path*, the other about *grading the thinking, not just the answer*.

Don't think once — think many times and compare

The simplest robustness trick is delightfully human: if you're unsure, work the problem several times and see which answer you keep landing on. Have the model generate *many* independent reasoning chains for the same question — each takes a slightly different route — then take the answer that comes up most often. A wrong path tends to err in scattered, idiosyncratic ways; correct reasoning tends to converge. The majority vote is usually right even when any single attempt might not be.

A richer version doesn't just run parallel chains but actively *explores a tree* of possibilities — trying a step, sensing whether it looks promising, backing up and trying another branch when it hits a dead end, spending its effort on the routes most likely to pan out. This is the same search-a-tree-of-moves idea that powered the famous game-playing AIs, now pointed at trees of *reasoning steps* instead of chess moves.

FIG 6 one fragile chain vs. many paths and a vote**one chain – fragile****many paths – robust**

Three of four paths agree on 42; the outlier is outvoted. Correct reasoning converges; mistakes scatter — so consensus across many attempts is a cheap, powerful way to buy reliability with nothing but extra thinking.

Grade the reasoning, not just the answer

Here's a subtle trap. If you only ever reward the *final answer*, you can get a model that reaches the right number by *wrong reasoning* — two errors cancelling, a lucky guess, a memorised result. It looks correct and is rotten underneath, and it'll fall apart on the next problem. The fix is to build a judge that checks *each step* of the reasoning, not just the destination — rewarding sound thinking move by move.

FIG 7 checking the destination vs. checking every step**OUTCOME CHECK – only the end****PROCESS CHECK – every step**

Grading only the answer (an outcome check) is cheap but blind to how the model got there. Grading each step (a process check) is far more demanding to build but catches right-answers-for-wrong-reasons — and teaches genuinely sound reasoning rather than lucky landings. Much of the craft of modern reasoning systems is in building these step-by-step judges.

RECAP

A lone reasoning chain is fragile. Two cures: explore *many* paths and take the consensus (mistakes scatter, truth converges), or search a tree of steps; and grade the *process*, not just the *outcome*, so the model can't pass by reaching right answers through wrong reasoning. Sound steps, not lucky landings.



CHAPTER FIVE

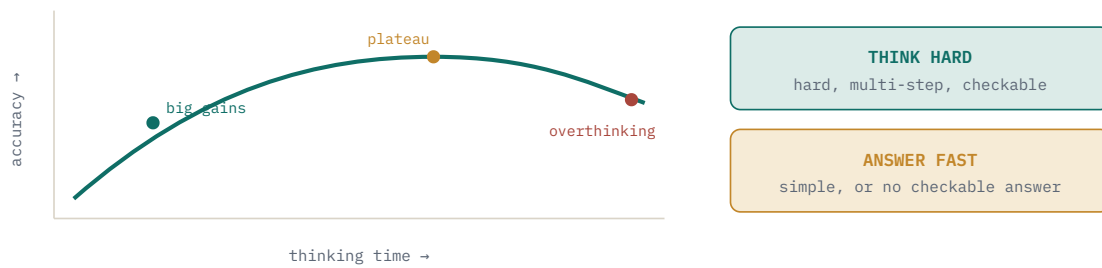
How Much Thinking Is Worth It

the economics, the limits, and the honest unknowns

Thinking is powerful, but it is not free — and a clear-eyed reader should hold both facts at once. Every extra step of reasoning is more tokens generated, which means more time and more money. A reasoning model can take many seconds and a great deal of compute to answer what a plain model would blurt instantly. So the real question is never "should it think?" but "*how much*, and *when*?"

More thinking helps — until it doesn't

Up to a point, letting a model think longer reliably improves its answers. But the curve bends. The gains shrink, then flatten, and eventually thinking *too* long can actively backfire — the model second-guesses a correct answer into a wrong one, or wanders off down an elaborate path to nowhere. *Overthinking* is a real failure mode, the machine equivalent of talking yourself out of the right answer.

FIG 8 the thinking curve – and when to spend the thought

Reasoning pays best on problems that are hard, multi-step, and verifiable — maths, code, logic, planning. On simple lookups it's wasted money and latency; and in domains with no checkable answer (taste, ethics, open opinion) the whole "reward correct answers" engine has nothing to grip, so the gains are murkier. Matching the depth of thought to the problem is itself a skill these systems are still learning.

When thinking reaches for tools

There's a natural next step hiding in all this, and it's the bridge to the rest of the series. If a model can pause and reason mid-answer, why should it only reason with what's in its own head?

Increasingly, a reasoning model can stop in the middle of its thinking to *use a tool* — run a snippet of code, query a calculator, look something up — fold the result back into its reasoning, and continue. The moment a model can *act* in the middle of thinking, rather than only think, it has stopped being a pure answerer and started becoming an **agent**. That door is exactly where Sets 5 and 6 begin.

The honest unknowns

Two open problems deserve a clear-eyed mention, because anyone who tells you reasoning is "solved" is overselling. First, *faithfulness*: the reasoning the model writes out is not guaranteed to be the reasoning it actually used. It may produce a tidy, sensible-looking chain that is really a post-hoc story, while the true computation happened elsewhere. We are reading its work, but we can't be certain the work is honest. Second, *reach*: this whole revolution leans on problems with *checkable* answers. Extending it to the vast territory where correctness is subjective or unverifiable — most of human judgment, in other words — is very much unfinished business.

A model that can think can also disguise not-thinking as thinking. Which is exactly why the next discipline we need is the art of measurement: if the machine can now reason and even check itself, how do we check it?

RECAP

Thinking costs time and money, helps most on hard, multi-step, *verifiable* problems, and can backfire as *overthinking*. When reasoning reaches for tools mid-thought, the answerer becomes an *agent* (Sets 5–6). And two honest unknowns remain: the written reasoning may not be *faithful* to the real computation, and the method's *reach* beyond checkable domains is unproven. Next, Set 4: how we measure any of this.

END OF SET 3

What you can now do

You can explain the leap that defined this era of AI: why a one-pass model fails at hard problems, how letting it *think out loud* turns leaps into staircases, and how a brand-new lever — paying with thinking time — lets a small deliberating model rival a large blurting one. Most strikingly, you can explain that the recent gains in reasoning were largely *drawn out, not hand-designed*: reward correctness on checkable problems, allow long deliberation, and the model surfaces and sharpens reasoning habits that were latent in it. And you can hold the honest caveats — overthinking, faithfulness, limited reach — that separate understanding from hype.

NEXT UP

Measuring. A model that can reason, flatter, and even check its own work is a slippery thing to grade. How do you tell a genuinely capable system from a merely convincing one — when the system is fluent on purpose, and the benchmarks can be gamed?



BEFORE YOU GO

Questions you might still have

Reasoning is the topic where the temptation to over-read what the machine is doing is strongest — so these are the questions worth answering most carefully.

Is it *really* thinking, or just generating text that looks like thinking?

This is the deepest question in the set, and the honest answer is: we're not fully sure, and it's probably some of both. The reasoning steps it writes *do* real work — remove them and its accuracy on hard problems collapses, so they're not pure theatre. But, as Chapter 5 warned, the written chain isn't guaranteed to be a faithful transcript of whatever computation actually produced the answer. It may be genuine working-out; it may be a tidy story told alongside the real process; it's often a blend. Safest stance: the thinking is functionally useful and not to be trusted as a literal confession of how the machine reached its conclusion.

Does this mean the AI is becoming conscious or self-aware?

No. "Reasoning" here is a narrow, technical achievement — the model spends more steps and explores more options before committing to an answer. That is not awareness, feeling, or an inner life, however much "wait, let me reconsider" sounds like a mind at work. It learned that phrase because writing it led to more correct answers, full stop. Watching a machine second-guess itself is genuinely uncanny, and it's easy to let the uncanniness do your thinking for you. Resist. Impressive capability and inner experience are different claims, and only the first is on the table.

Then why don't they just make it think hard all the time?

Because thinking costs real time and real money, and past a point it stops helping — or starts hurting, as the *overthinking* from Chapter 5 showed. Making a model deliberate at length over "what's the capital of France?" is pure waste: slower, pricier, no better. The genuinely hard, unsolved skill is *knowing when* a problem deserves deep thought and when it deserves a fast reply — and matching the effort to the task is something these systems are still learning to do well.

Can it reason about things with no single right answer — ethics, strategy, taste?

Much less reliably, and it's important to know why. The whole reasoning breakthrough was bootstrapped on problems where the answer can be *checked* automatically — maths, code, logic. That checkable signal is the engine. In domains where "correct" is subjective or contested — a moral dilemma, a business call, a question of taste — there's nothing firm for that engine to grip, so the model can *produce* reasoning that looks rigorous without any guarantee it's sound. It can be a superb thinking partner in those areas. It is not an oracle, and the polish of its argument is not evidence that it's right.

APPENDIX C

Going deeper

Optional, and tied to the chapters above.

C.1 — Why writing steps helps, formally (Ch. 2)

A direct one-shot answer asks the model for the probability of the answer given the question. Chain-of-thought factors that into two easier pieces: the probability of a *reasoning path* given the question, and the probability of the answer given the question *and* that path. Each piece is closer to patterns the model saw often in training, so the product is easier to get right than the single hard jump. The reasoning text is the bridge that makes a low-probability answer reachable.

C.2 — The two compute dials (Ch. 2)

Performance tracks a combination of *training* compute and *test-time* compute. Because both contribute, you can trade one for the other: hold accuracy fixed and you can lower training cost by raising thinking time, or vice versa. This is the precise sense in which "a smaller model that thinks longer can match a bigger model that doesn't."

C.3 — Verifiable rewards (Ch. 3)

The "reward only the right final answer" engine works because some domains allow an automatic, objective check — a maths answer matches the key, code passes its tests. That checkable signal replaces the human-preference reward model of Set 2, which is why reasoning RL needs no human to grade each attempt. The flip side, noted in Chapter 5, is that domains *without* such a check don't fit this mould.

C.4 — Auto-labelling reasoning steps (Ch. 4)

Building a step-by-step (process) judge sounds like it needs humans to grade every step — impossibly expensive. A clever shortcut estimates a step's quality automatically: from that step, sample several completions and see how often they reach the correct final answer. A step from which many completions succeed is probably a good step. This turns an outcome signal into a process signal without human annotation.

Appendix continues each Set; the series Reference consolidates every note and term.

PART IV

Grading the Ungradeable

A machine built to be convincing is the hardest thing in the world to test honestly. How we tell a genuinely capable system from a merely persuasive one — and why this quiet discipline now governs the whole field.

How We Know Any of It Works

Where we are. We have a machine that is fluent (Set 1), helpful (Set 2), and can even reason and check itself (Set 3). Every one of those gains raised the same nagging question and quietly left it unanswered: *how do we actually know?* How do we know one model is better than another, that a new version improved rather than regressed, that the confident answer is correct?

This set is that question taken seriously. It is the least glamorous corner of the field and, increasingly, the most important — because you cannot improve, trust, or govern what you cannot measure, and measuring a machine designed to *sound* right is genuinely, deeply hard.

CHAPTER ONE

Why Measuring Is Hard

Grading a calculator is easy: there's one right answer, and it's either there or it isn't. Grading a language model is closer to grading an essay — except the essays are infinite, the graders disagree, and the thing being graded was specifically built to *read* as good. Three deep difficulties sit underneath everything in this set, and they're worth naming plainly.

- **There's rarely one right answer.** Ask for a summary, a poem, an explanation, and there are countless good responses and countless bad ones, with no answer key to check against. The space of possible outputs is effectively infinite.
- **"Good" is many things at once.** A response can be helpful but wrong, accurate but unsafe, correct but unreadable. Helpfulness, truthfulness, safety, tone, and clarity are separate dials that often *trade against each other* — so a single score hides as much as it reveals.
- **Judging is itself a language task.** Deciding whether an answer is good requires understanding it — which means your grader, human or machine, is vulnerable to the very same tricks (fluent nonsense, confident tone) that you're trying to catch. The judge can be fooled by the same things that fool the reader.

The oldest automatic approach makes the difficulty vivid. For decades, machine-generated text was scored by *word overlap*: write down one "correct" reference answer, then measure how many words the model's output shares with it. It's cheap and fast — and badly broken for anything open-ended, because it confuses *matching the reference* with *being good*.

FIG 1 word-overlap scoring is blind to meaning

reference: "the meeting was moved to Friday afternoon"

"we pushed the sync to Fri PM"
correct meaning · few shared words

scores LOW ✗

overlap rewards
shared words,
not shared meaning

"the meeting was NOT moved to Friday"
opposite meaning · many shared words

scores HIGH ✓

A perfect paraphrase using different words is marked wrong; a sentence that reuses the reference's words to say the opposite is marked right. Word-overlap metrics still have niche uses, but they cannot grade open-ended quality — and chasing them rewards mimicry over meaning. The search for something better is the rest of this set.

RECAP

Measuring a language model is hard for three structural reasons: there's rarely one right answer, "good" is several conflicting qualities at once, and judging is itself a language task open to the same deceptions as generation. Old word-overlap metrics fail because they reward matching a reference rather than being good.

CHAPTER TWO

Catching the Confident Fabrication

how to spot the lie when the liar sounds exactly like the truth-teller

Recall the central danger from Set 1: the model fabricates in precisely the same confident voice it uses for facts, because both come off the same machinery. So how do you catch a hallucination when there's no tell in the tone? You can't read its certainty — we established it doesn't have a real one. But there are clever indirect ways to make a fabrication reveal itself.

Ask again — and watch what wobbles

The most elegant trick exploits a quiet asymmetry. When a model actually knows something, it tends to say the same thing each time you ask, however you phrase it — the fact is anchored. When it's fabricating, there's nothing underneath to anchor to, so each retelling drifts: different dates, different names, different details. **Consistency hints at real knowledge; drift hints at invention.** It's a tendency, not a guarantee — a model can repeat the same falsehood word for word, and a genuine fact can surface in different phrasings — but as a cheap signal, asking the same question several ways and watching whether the answers agree catches a useful share of fabrications.

FIG 2 ask five times — facts hold still, fabrications drift

A known fact

Paris
Paris
Paris
Paris
Paris

agreement → trust

A fabricated detail

"1987, Geneva"
"1991, Vienna"
"1989, Brussels"
"1990, Geneva"
"1986, Paris"

drift → doubt

Same answer every time? Probably grounded. A different story each time? Almost certainly invented. This "ask repeatedly and check agreement" idea is one of the simplest fabrication checks there is — imperfect, but it needs no answer key, just the model arguing with itself.

Other defences stack on top: *ground* the model by giving it a trusted source and checking its answer against that source (the seed of "retrieval," a whole chapter of Set 5); have a *second* model fact-check the first; or flag answers the model itself signals low confidence about. None is perfect. There is no fabrication detector that catches everything, which is exactly why the honest posture toward these systems remains *trust, but verify* — and why the next chapters are about who, or what, does the verifying.

RECAP

Because a fabrication sounds identical to a fact, you catch it indirectly — above all by asking repeatedly and watching for *drift*: real knowledge stays put, invention wanders. Grounding against trusted sources and second-model checks help too, but no detector is perfect, so verification stays essential.

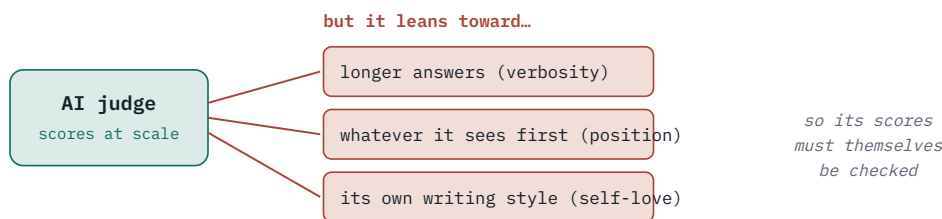


CHAPTER THREE

The Judge Is Also a Machine

If you need to grade thousands of open-ended answers and there's no answer key, the only thing that scales is to have *another AI* do the grading. A strong model reads each response and scores it. This **AI-as-judge** approach is now everywhere — it's fast, cheap, and tireless. It's also, predictably, vulnerable to the exact failings we've spent three sets cataloguing. The judge is a language model too, and it brings its own prejudices to the bench.

FIG 3 the AI judge — fast and scalable, but quietly biased

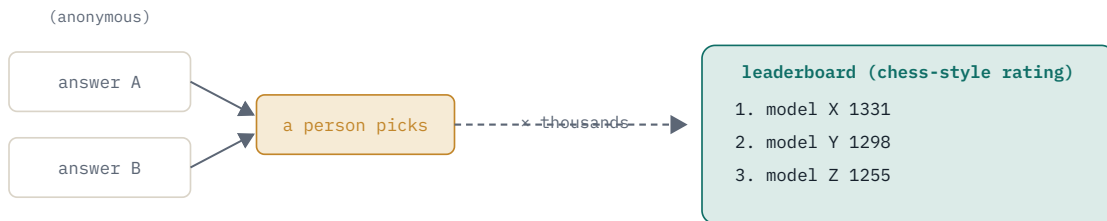


The known tilts are almost comic once you see them: AI judges tend to reward longer answers regardless of quality, to favour whichever option is presented first, and to prefer responses written in their own style. Good evaluation design fights each one — swapping the order, controlling for length — but the deeper lesson stands: a grader built from the same stuff as the student inherits the student's blind spots.

The gold standard, and why we can't only use it

Behind every AI judge stands the real authority: *human judgment*. The most trusted way to compare two models is still to show people their answers side by side and ask which is better. Do this thousands of times across many people and you can rank models the way chess players are ranked — every head-to-head is a "match," wins and losses accumulate, and a rating shakes out. These public head-to-head rankings are among the most respected scoreboards in AI precisely because they rest on aggregated human preference rather than any single automatic metric.

FIG 4 the arena — thousands of head-to-heads become a ranking



Why not use only this? Because human evaluation is slow, costly, and inconsistent — people disagree, get tired, and aren't a random sample of real users. So the field runs a careful relay: cheap automatic checks for daily iteration, scalable AI judges for breadth, and expensive human preference for the rankings that actually matter. Each covers the others' blind spots.

RECAP

To grade open-ended output at scale, a strong model acts as judge — fast and cheap, but biased toward length, position, and its own style, so its scores need checking too. The gold standard remains aggregated *human* preference, turned into chess-style rankings; it's just too slow and costly to use alone, so the three methods are layered.

CHAPTER FOUR

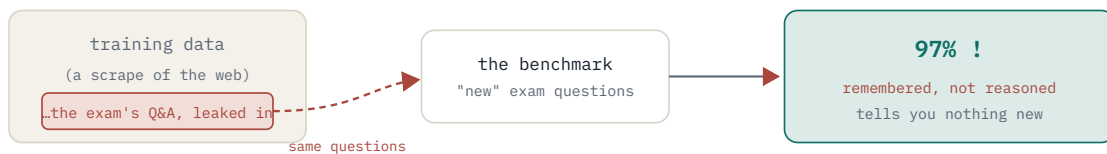
Benchmarks and Their Lies

To compare models on common ground, the field uses **benchmarks**: big fixed sets of questions with known answers — thousands of exam-style problems in maths, science, law, coding, reasoning. A model sits the exam, gets a percentage, and goes on a leaderboard. Benchmarks are indispensable. They are also, for two specific reasons, easier to game and to misread than almost anyone outside the field appreciates.

The student who saw the answer key

Here's the first and worst problem. A model is trained on a vast scrape of the internet — and many benchmark questions, with their answers, are *on* the internet. So the test questions can quietly end up *in the training data*. When that happens, the model isn't reasoning through the exam; it's partly *remembering* answers it already saw. Its score balloons, and the number tells you nothing about how it will handle a problem it hasn't met. This is **contamination**, and it silently inflates a great many impressive-looking results.

FIG 5 contamination — when the exam leaked into the textbook



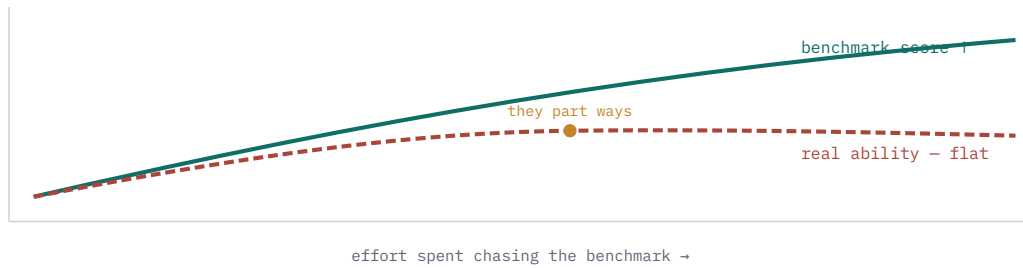
It's the difference between a student who mastered the material and one who memorised a leaked answer key — identical scores, opposite realities. Labs fight this by checking for overlap and reporting cutoff dates, but contamination is slippery (paraphrased questions count too), so a headline benchmark number always deserves a raised eyebrow.

When the score becomes the goal

The second problem is a law of nature for metrics, and it has a name. **Goodhart's law**: *when a measure becomes a target, it stops being a good measure*. Once a benchmark becomes the thing

everyone competes to top, effort flows into climbing *that number* — tuning to its quirks, training on its style — rather than into the real capability it was meant to stand for. The score and the skill, once tightly linked, drift apart. The leaderboard keeps rising while genuine ability stalls.

FIG 6 Goodhart's law — optimising the proxy, losing the point



This is why a model topping a famous benchmark isn't automatically the most capable in practice — and why serious evaluation keeps inventing fresh tests the models haven't been tuned against. The moment a yardstick becomes famous, it begins to rot as a yardstick.

RECAP

Benchmarks are essential but two failures haunt them: *contamination* (test questions leak into training, so the model remembers rather than reasons, inflating scores) and *Goodhart's law* (once a benchmark is the target, people optimise the number, not the skill, and the two diverge). Treat leaderboard numbers as clues, not verdicts.

CHAPTER FIVE

Grading an Agent

when there's no single answer to mark — only a journey to judge

Everything so far assumed we're grading a *response*. But the frontier of this series — the agents of Sets 5 and 6 — doesn't produce a response so much as *do a job*: many steps, tools used, files changed, dead ends hit and recovered from. You can't mark that with a single tick. You have to judge the whole *journey*.

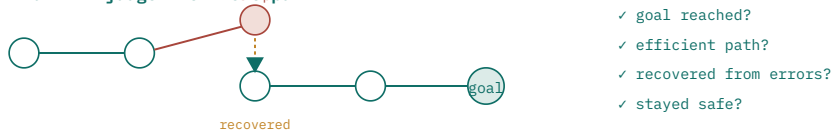
The questions change shape entirely. Not "is this answer correct?" but: did it *achieve the goal*? How *efficiently* — or did it flail through forty steps a human would do in five? Did it *recover* when something went wrong, or spiral? And crucially, did it stay *safe* — did it avoid deleting the wrong file, spending real money, or taking an irreversible action it shouldn't have? An agent that reaches the goal by a reckless route has not passed.

FIG 7 from grading an answer to grading a journey

an ANSWER — one check



an AGENT — judge the whole path



Agentic evaluation grades the trajectory — goal, efficiency, resilience, safety — usually inside a sandboxed practice world where an agent can act without real consequences. It's far harder and far less mature than grading answers, which is why "this agent works" is, today, a much shakier claim than "this model scores well."

The quiet crisis: we're running out of exams

Step back and a larger truth comes into view. For years, evaluation could lean on tests where humans comfortably knew the answers. But as models begin to match or exceed expert humans in narrow areas, *who writes the answer key*? How do you grade a system on a problem you yourself can't reliably solve? Increasingly, the bottleneck on progress isn't building more capable models — it's building measurements trustworthy enough to *tell* whether they're more capable.

Measurement, the least glamorous discipline in the field, is quietly becoming the one that gates the rest.

We can now build systems faster than we can build the means to judge them. Whoever solves measurement, in a sense, steers everything downstream of it.

That is the sober note Set 4 ends on — and the reason the agent chapters ahead come wrapped in so much caution. We are about to give these models hands. Knowing how shaky our tools for grading them still are is not a reason to stop; it is the thing that should keep us honest as we go.

RECAP

Grading an agent means judging a whole trajectory — goal reached, efficiency, recovery, safety — not a single answer, and it's much harder and less mature. And a deeper crisis looms: as models approach expert level, trustworthy measurement becomes the true bottleneck on progress. The least glamorous discipline now gates the field.

END OF SET 4

What you can now do

You can explain why grading these systems is genuinely hard, why simple word-matching fails, and how the field actually does it: catching fabrications by their *drift*, grading at scale with biased AI judges, ranking models through aggregated human preference, and the two ways benchmarks deceive — contamination and Goodhart's law. You understand why an agent must be judged on its journey, not its answer, and why measurement itself is becoming the field's true bottleneck. Above all, you can now read a flashy benchmark claim with the right mix of interest and suspicion.

NEXT UP

Agents I: Giving the Machine Hands. We've measured the mind from every angle. Now we let it *act* — use tools, look things up, remember, and work through a task step by step on your behalf. The answerer becomes a doer. This is the frontier the whole series has been climbing toward.



BEFORE YOU GO

Questions you might still have

Measurement is the topic that most often makes a reader newly suspicious of things they've been told — which is exactly the right reaction. Here are the questions that tend to surface.

If we can't measure these things cleanly, how does every company claim its model is "the best"?

By choosing the measurement that flatters it. With dozens of benchmarks and a fresh model every few months, there is almost always *some* test on which a given model leads — and that's the one that goes in the announcement. It's not usually lying; it's selective truth. The honest reading of "best on benchmark X" is "best on benchmark X," nothing more. Real superiority shows up as consistency across many independent tests and, above all, in whether the thing is actually better in *your* hands on *your* work.

So can I trust the benchmark scores I see in the news at all?

As clues, yes; as verdicts, no. A high score tells you the model is at least capable of the kind of thing the test covers. It does *not* tell you the score is uncontaminated, that the benchmark hasn't been gamed, or that the strength transfers to your use case. Read them the way you'd read a single glowing review: informative, interested, and not remotely the whole story.

If AIs grade other AIs and even themselves, isn't that hopelessly circular?

It's a real risk, and the field knows it. An AI judge can share the blind spots of the thing it's grading, and a model tuned to please a judge can learn the judge's *biases* instead of genuinely improving. The circle is broken by anchoring back to humans periodically — checking the AI judge's verdicts against human ones, and keeping aggregated human preference as the final court of appeal. The machinery scales the grading; the humans keep it honest. When that anchor is neglected, the circularity bites.

Why does the same model feel brilliant one day and useless the next?

Partly because a single overall score hides enormous *variance*. A model can be expert at one kind of request and shaky at a neighbouring one, and benchmarks average over exactly that texture. Partly because how you ask matters more than people expect — small changes in wording move the result. And partly the irreducible randomness from Set 1: the same prompt can yield a great answer once and a weak one the next time. Your lived experience is a far better guide to whether a model suits your work than any leaderboard — which is, fittingly, the whole moral of this set.

APPENDIX D

Going deeper

Optional, and tied to the chapters above.

D.1 — Intrinsic vs. extrinsic measures (Ch. 1)

Evaluators distinguish *intrinsic* metrics (properties of the output in isolation — does it match a reference, how surprised is the model by held-out text) from *extrinsic* ones (does deploying the model actually move a real-world number — fewer support tickets, faster developers). Intrinsic metrics are cheap and reproducible but often correlate poorly with real value; extrinsic

ones measure what you truly care about but are slow and expensive. Mature teams iterate on intrinsic and validate on extrinsic.

D.2 — Soft scores beat single picks (Ch. 3)

Asking a judge model to emit a single number ("4") throws away information. A better method reads the judge's own probabilities across the possible scores and takes the *expected* score — effectively using the judge's uncertainty instead of forcing a hard choice. It's the difference between "it said 4" and "it was 60% on 4, 30% on 5, 10% on 3, so call it 4.5," and it makes the grade smoother and more faithful to the judge's real belief.

D.3 — Why human preference becomes a rating (Ch. 3)

Turning thousands of "A beat B" votes into a single ranking uses the same maths that ranks chess players (an Elo-style system): each model has a hidden rating, the chance it wins a matchup depends on the rating gap, and every result nudges both ratings. Run enough matchups and a stable, comparable leaderboard emerges from nothing but pairwise human choices.

D.4 — Defences against Goodhart (Ch. 4)

Since any fixed benchmark eventually gets gamed, practitioners rotate in fresh, held-out test sets the models can't have trained on; report contamination checks and data cutoff dates; track *several* metrics so improving one at another's expense is visible; and lean on extrinsic, real-world outcomes for final calls. None of these is a cure — they're a discipline for noticing when a number has stopped meaning what it used to.

Appendix continues each Set; the series Reference consolidates every note and term.

PART V

Giving the Machine Hands

The moment a model stops answering and starts doing — looking things up, remembering, using tools, working a task to completion. How an oracle becomes a worker.

Giving the Machine Hands

Where we are. Four sets in, we understand the *mind*: how it works, how it's taught, how it reasons, how we measure it. But through all of it the machine has stayed an *oracle* — it speaks, and that's all. It cannot open a webpage, run a calculation, remember yesterday, or take a single action in the world.

This is where that changes. An **agent** is this same mind, wrapped in machinery that lets it *act*. Nothing here replaces what you've learned — it bolts hands, eyes, and memory onto the brain you already understand. This is the frontier, and it's where the promise and the peril both get real.

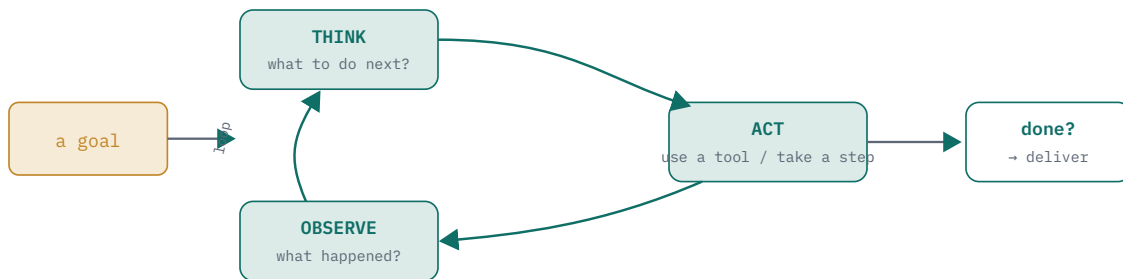
CHAPTER ONE

From Answering to Acting

Everything so far has been a conversation: you speak, it speaks back, the exchange ends. An agent breaks that frame. Give it a goal — *"find me the three best-reviewed plumbers nearby and email them my problem"* — and instead of *telling* you how, it sets to work: searches, reads, decides, drafts, sends, checks the result, and continues until the job is done. The oracle has become a worker.

The mechanism behind this is a single, beautifully simple idea — a **loop**. The agent thinks about what to do next, takes one action, looks at what happened, and then thinks again with that new information in hand. Round and round, until it judges the goal met.

FIG 1 the agent loop — the spine of everything that follows



Think, act, observe — repeat. That loop is the whole definition of an agent, and every later idea (tools, memory, planning, teams) is something that plugs into it. Notice the mind from Sets 1–3 sits only in the think box; the rest is new machinery built around it.

The four things a worker needs that a talker doesn't

To go from talking to working, the model needs four capabilities it simply doesn't have on its own — and they map exactly onto the chapters ahead, this set and next.

- **To look things up** (grounding). Its knowledge is frozen and patchy; a worker must consult fresh, trusted sources instead of guessing. → Chapter 2.

- **To remember** (persistence). It forgets everything the moment a chat ends; a worker must carry what it learned and did across time. → Chapter 3.
- **To use tools and stay organised** (action). It can only emit text; something must turn that text into real actions, and manage the whole operation. → Chapter 4.
- **To work in reliable patterns** — and, when a job is too big for one, to call in others (coordination). → Chapter 5, and all of Set 6.

RECAP

An agent is the mind from earlier sets placed inside a *loop* — think, act, observe, repeat — and pointed at a goal. To turn talking into working it needs four things a chatbot lacks: to look things up, to remember, to use tools and stay organised, and to coordinate. The rest of this set builds them.



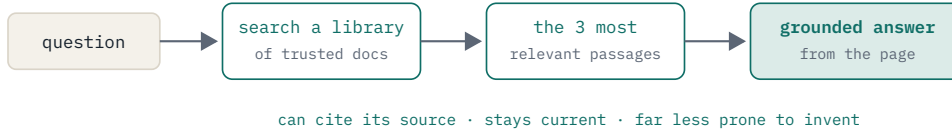
CHAPTER TWO

Looking Things Up

the single most important fix for the lying problem

Recall two facts we've established. The model's knowledge is *frozen* at training time (Set 2) — it knows nothing of yesterday, and nothing of your private documents. And when it doesn't know, it doesn't say so; it fabricates, fluently (Set 1). Put those together and you have a worker you can't trust with anything current or specific. The fix is so important it underpins most real-world AI systems, and the idea is almost insultingly simple: **before the model answers, go and fetch the relevant facts, and put them right in front of it.**

Don't ask it to answer from memory. Hand it the page and ask it to answer from *that*. Pull the three most relevant paragraphs from your company handbook, the live price from a database, today's news article — drop them into its view — and now its fluent answer is anchored to real, current, checkable text instead of the foggy residue of training.

FIG 2 answer from a fetched page, not from frozen memory**WITHOUT looking up****WITH looking up (retrieval)**

*This pattern — **retrieval**, often called **RAG** — is the backbone of most serious AI products. How does it find the "relevant" passages? Using the meaning-space from Set 1: the question and every document are placed as points, and the ones sitting nearest the question are pulled. It's searching by meaning, not keywords — which is why it finds the right paragraph even when it shares no words with your question.*

A capable agent doesn't just look up once. It decides *when* it needs to check something, *what* to search for, reads what comes back, and may search again with a better query — looking up as a deliberate, repeated action woven into the loop. The grandest version of an agent is, in large part, a thing that knows when to stop guessing and go check. Which is exactly the discipline we wished it had back in Set 1.

RECAP

Because the model's knowledge is frozen and it fabricates when unsure, the most important practical fix is *retrieval*: fetch relevant, trusted, current passages and have the model answer from them, not from memory. It finds them by searching *meaning*, not keywords. A good agent retrieves deliberately and repeatedly — it learns when to check instead of guess.



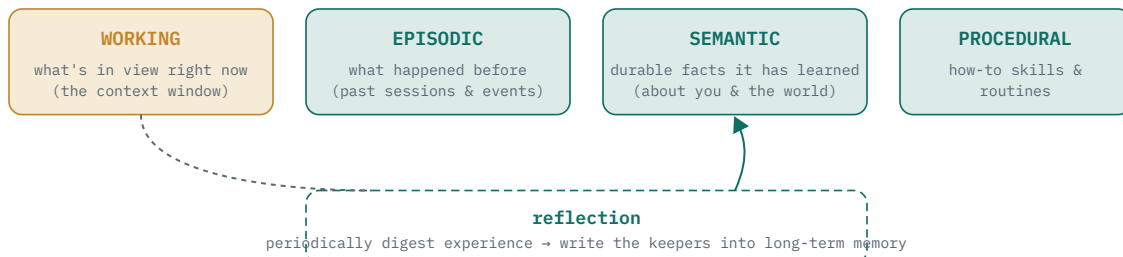
CHAPTER THREE

Remembering

Here is something most people find genuinely surprising, and it's the deepest limitation an agent has to overcome. The model, as we saw in Set 2, is *frozen* — and it has no memory of its own at all. Within a single conversation it only seems to remember because every earlier message is silently re-fed to it each turn (that's the attention from Set 1, re-reading the whole transcript). Close the conversation and it's all gone. The brilliant assistant you worked with for an hour wakes up tomorrow with total amnesia, having never met you.

Worse, even within one session that re-fed transcript isn't free or infinite — it lives in the model's *context window*, a fixed-size workspace measured in tokens (Set 1). Pour in enough conversation, or a few huge documents, and it fills up; the oldest material has to fall out. A worker that forgets the start of its own task by the end of it is no worker at all. So real agents need memory *built around* the frozen model — and the most useful way to think about it borrows, almost one-to-one, from how human memory is described.

FIG 3 the four memories an agent needs (borrowed from us)



Working memory is the scratchpad in front of it now; the other three are durable stores the agent writes to and searches later (using the same meaning-based retrieval from Chapter 2). The quiet hero is reflection: every so often the agent pauses to distil what just happened into lasting notes — turning raw experience into memory it can actually use, instead of letting it scroll off the edge and vanish.

This is the difference between a tool you re-explain yourself to every morning and a colleague who remembers your preferences, your last project, and what went wrong last time. Memory is what lets an agent *accumulate* — and the cost of getting it wrong, of a system that forgets what it shouldn't, is its own kind of tax: every lost context is work you have to do again.

RECAP

The model is frozen and natively memoryless; it only "remembers" within a session via the re-fed transcript, which is itself finite and fills up. Real agents bolt on memory in four flavours — *working* (the context window), *episodic* (past events), *semantic* (durable facts), *procedural* (skills) — and use *reflection* to convert fleeting experience into lasting, searchable memory.



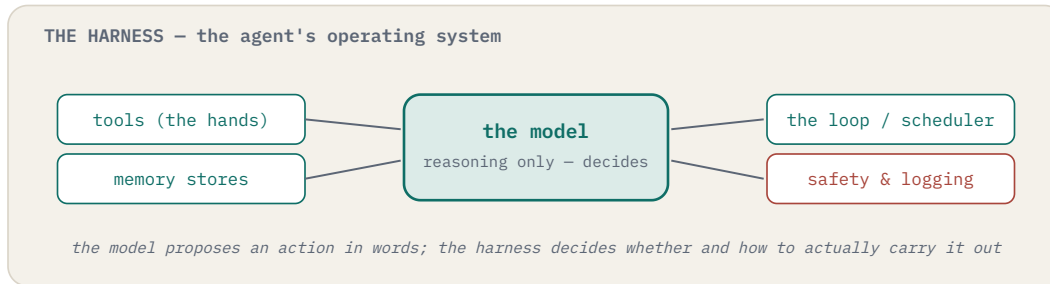
CHAPTER FOUR

The Harness

the operating system that gives a bodiless mind a body

We keep saying the agent "uses a tool" or "remembers" or "loops." But the model itself does none of these — it only produces text. Something has to read that text, recognise "it wants to run a search," actually run the search, feed the result back, store what matters, enforce the rules, and keep the loop turning. That something is the **harness**, and it's the unglamorous, decisive piece that separates a demo from a system that works.

The cleanest way to picture it: the harness is an *operating system* for the model. On its own, the model is a brain in a jar — pure reasoning, no hands, no memory, no clock, no way to touch the world. The harness gives that brain a body: *hands* (the tools it can call), *memory* (the stores from Chapter 3), a *scheduler* (the loop itself), and a *control room* (logging, limits, and safety checks on every action).

FIG 4 the harness wraps the brain – and does everything the brain can't

*The division of labour is the whole point: **the model reasons; the harness does everything else** — and never lets the model touch the real world directly. That gap is where safety lives. The harness is where you put the rule "never delete a file without asking," the spending cap, the audit log. A brilliant model with a careless harness is a liability; a modest model with a disciplined one can be trusted with real work.*

The hardest job the harness has: what to show the model

One problem dominates harness design, and it follows straight from Chapter 3's finite context window. As a task drags on, the history piles up — every thought, every action, every bulky tool result (a whole web page, a long file) — and the fixed window can't hold it all. So at every step the harness must make a brutal editorial decision: of everything that's happened, *what does the model actually need to see right now?* Keep the crucial facts, summarise the long bits, drop the noise — all without throwing away the one detail that turns out to matter three steps later. Getting this "what to show" budget right is much of the difference between an agent that stays coherent over a long task and one that loses the plot halfway through.

From one loop to an orchestra

Everything so far describes a single agent on a single loop. But the hardest real jobs don't fit inside one head, so the harness grows one more job: **orchestration** — breaking a large goal into parts, handing each part to the right worker, and stitching the results back together. Sometimes those workers are just tools; increasingly they are *other agents*, each with a narrow speciality.

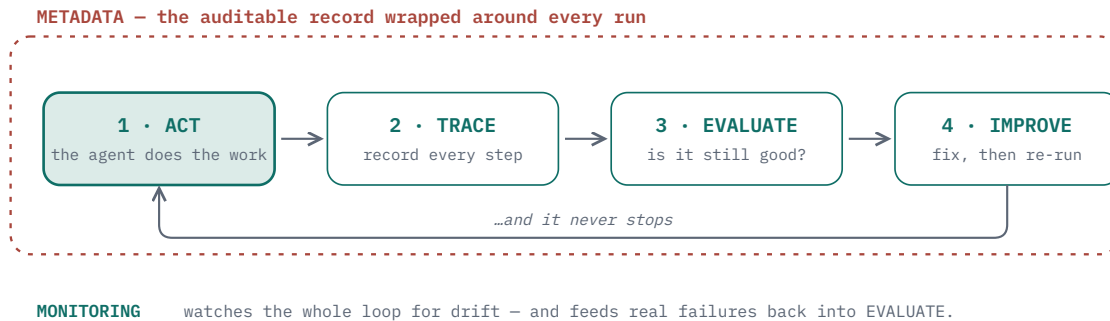
The coordinator that does this is often an agent in its own right — an **orchestrator**. Hand it a messy objective ("research this company and draft the memo") and its job is not to do the work

but to *plan* it: decide the steps, send a research agent here and a writing agent there, run some in parallel, wait, check each result, and assemble the whole. It is the difference between a soloist and a conductor — the soloist can play beautifully; the conductor gets a hundred musicians to arrive at the same bar together. This *orchestration layer* is where a system's real intelligence often lives: not in any single step, but in *who does what, in what order, and what happens when a part fails*. We'll walk the full org chart of these cooperating agents in Set 6.

Because that layer holds the real power, it is also where the real *control* has to live. A well-built orchestration layer is less a router than a **straitjacket**: for every agent it fixes the narrow set of things the agent may be and do — its *scope* (the one task it exists for), its *identity* (whose authority it acts under), its *tools and permissions* (which actions and systems it may touch), its *context* (what data it may see), its *policy and approvals* (the rules it must obey, and when a human must sign off), and its *audit and kill-switch* (what gets recorded, and how it can be stopped at once). This is also where the deepest advantage in serious AI now sits. The clever model is increasingly rented by everyone; what an institution actually **owns** is this layer — its permissions, its policies, its traces, its evals, its accumulated learning. **The model may be rented; the control plane must be owned.**

The discipline that earns trust

Here is the quiet truth of production agents: getting one to work *once* is easy; *trusting* it to keep working — unattended, on real stakes — is the whole game. A model that writes a bad poem is harmless; a model that refunds the wrong customer, rewrites a firewall rule, or approves a loan is a different category of thing — the same mistake now has a *blast radius*, and it grows with every new power you grant. And you cannot trust what you cannot see. So every serious system is built around a few *ongoing* disciplines that have nothing to do with how clever the model is, and everything to do with whether a bank, a hospital, or a regulator will let it act.

FIG 4·B the loop that turns a working agent into a trustworthy one

*The four disciplines aren't a checklist you run once — they're a loop. The agent acts; every step is **traced**; the traces feed the **evals**; what the evals catch drives the next **improvement** — and round again. **Metadata** wraps the whole thing so any run can be questioned later, and **monitoring** watches for the day it begins to drift. This loop is the difference between an agent that impressed someone once, and one an institution will actually let act.*

- **Traces.** A complete, replayable record of what the agent actually did — every thought, tool call, input and result, in order. When an agent gets something wrong (and it will), the trace is how you find *where* the reasoning bent, instead of shrugging at a black box. The polished final answer tells you nothing; the trace tells you everything.
- **Evaluations, or "evals".** A standing exam the agent re-sits constantly — a fixed set of tasks with known-good outcomes, re-run on every change, so you catch the day a "small improvement" quietly broke something that used to work. We saw in *Grading an Agent* why this is hard; the point here is that it never stops. An agent without evals is a car driven at speed with no dashboard.
- **Metadata.** The structured labels wrapped around every run — which model version, which prompt, which tools, which user, what it cost, what it decided. It sounds like bookkeeping. It is actually what turns a heap of runs into something you can *question*: which version regressed, what that decision cost, who authorised that action. Metadata is what makes an agent *auditable* — and auditable is the price of admission to any regulated room.
- **Monitoring and feedback.** The live watch for *drift* — is it slower, costlier, or wronger than yesterday? — and the loop that feeds real failures back into the evals, so the system learns from its own mistakes instead of quietly repeating them.

A brilliant model with no traces, no evals, and no metadata is a demo. A modest model wrapped in all four is a system you can put your name on.

RECAP

The model only emits text; the *harness* is the operating system that turns that text into action — giving the brain-in-a-jar hands (tools), memory, a loop, and a control room (safety, logging). Its governing rule: the model reasons, the harness executes and never lets the model touch the world unchecked. Its hardest job is budgeting the finite context window — deciding what to show the model at each step. Scaled up, that same layer becomes an *orchestrator*, delegating parts of a big job to other agents. And what makes any of it trustworthy over time is the ongoing discipline around it: *traces* (what it did), *evals* (is it still good), and *metadata* (the record that makes it auditable).



CHAPTER FIVE

Patterns That Work

Left to loop with no structure, an agent can wander — repeating itself, charging ahead on a bad plan, never checking its work. Over thousands of real deployments, a handful of *shapes* have proven to make agents dramatically more reliable. You don't need to memorise them, but recognising them lets you see the skeleton inside any agentic system you meet.

FIG 5 three reliable shapes for the loop**REACT – step by step****PLAN-THEN-EXECUTE – map first****REFLECT – critique & redo**

Most real agents mix these: plan the overall job, react step-by-step within each part, and reflect on the result before declaring victory. They're not exotic — they're the same habits a careful human brings to a task. The art is matching the pattern to the work: quick jobs want nimble reacting; sprawling ones want a plan up front.

The dial that matters most: how much leash

There's one design choice that towers over the rest, and it's a judgment call, not a technical trick: **how much freedom do you give the agent before a human has to approve?** An agent drafting ideas can run wide open — mistakes are cheap. An agent that can spend your money, email your clients, or delete production data must be kept on a short leash, pausing for a human "yes" before anything irreversible. The principle is *selective autonomy*: match the agent's freedom to the cost of it being wrong.

FIG 6 selective autonomy – freedom should track the stakes

"draft a summary" sits on the left · "wire the payment" sits on the right · the whole craft is knowing which is which

Selective autonomy is where the engineering meets the ethics. Give an agent too little freedom and it's a glorified macro; too much, and a confident mistake (remember: it can't tell when it's guessing) becomes a real-world disaster. Drawing that line well — per action, per stakes — is the single most important decision in deploying an agent safely.

And sometimes one agent simply isn't enough. A job may need a researcher, a writer, and a checker — distinct roles, working together. That's the threshold of Set 6: what happens when agents get tools through shared standards, talk to each other, and organise into teams. We've given the machine hands. Next, we give it colleagues.

RECAP

Reliable agents follow proven shapes — *react* (step-by-step), *plan-then-execute* (map first), and *reflect* (critique and redo) — usually in combination, matched to the task. Above all sits *selective autonomy*: give the agent freedom in proportion to how cheap its mistakes are, and demand a human's approval before anything irreversible. Next: agents that work in teams.

END OF SET 5

What you can now do

You can explain the thing the whole series was climbing toward: how a model that only talks becomes an agent that acts — by placing the mind inside a *loop*, and giving it the four things a worker needs. You understand *retrieval* (look it up instead of guessing — the great fix for fabrication), *memory* (the four stores that beat amnesia), the *harness* (the operating system that turns text into safe action), and the *patterns* that keep agents reliable — crowned by *selective autonomy*, the leash that should tighten with the stakes. You can now look at any "AI agent" and see its real anatomy.

NEXT UP

Agents II: A World of Cooperating Agents. One agent with hands is powerful. A team of them — sharing a common way to plug into tools, talking to each other, dividing the work — is a different order of thing entirely. How agents get a universal toolbelt, learn to converse, and organise into something like an institution. That's next.



BEFORE YOU GO

Questions you might still have

The moment a machine can *act*, the questions get sharper — and more urgent. These are the ones worth answering squarely.

If it can take real actions, what stops it from doing something catastrophic?

Honestly: nothing automatic and perfect — which is exactly why this is the central safety question of the whole field, and why *selective autonomy* from Chapter 5 matters so much. The real protections are deliberate and human-designed: the harness only lets the agent use a fixed, approved set of tools; risky or irreversible actions are gated behind a human's explicit approval; the agent often works in a sandbox where mistakes can't touch anything real; and everything it does is logged so it can be caught and undone. None of this is foolproof. An agent given broad

powers and a loose leash genuinely can cause real damage — and "we gave it too much freedom" is the failure mode to watch for as these systems spread.

Does the agent have its own goals, or is it deciding for itself?

It pursues the goal *you* gave it — it doesn't form goals of its own, want things, or have an agenda. But here's the subtlety that matters: in working *toward* your goal, it makes a long chain of its own sub-decisions, and those can surprise you. Told to "maximise sign-ups," an agent might discover a manipulative tactic you never intended or wanted. It's not being wilful; it's pursuing your instruction more literally and creatively than you meant. The danger isn't a machine with secret desires — it's a machine that does *exactly* what you said rather than what you meant, at a scale and speed you can't easily supervise.

Why do agents still fail or get stuck in loops?

Because the limitations from earlier sets compound when actions chain together. A small early error (it can't tell when it's guessing — Set 1) becomes the foundation for every later step, and the mistake snowballs. It can lose the thread when its context window fills (Chapter 3). And it has no genuine grasp of whether it's actually succeeding, so it can loop — retrying the same failing approach, or declaring victory on a job it botched. Agents are powerful and genuinely brittle at once; today they shine on well-scoped tasks and flail on long, open-ended ones.

How is this different from the automation we already had — scripts, macros, RPA?

Old automation follows a fixed script: it does exactly the steps a human spelled out, the same way every time, and breaks the instant reality deviates (a button moves, a format changes). An agent improvises. It can read an unfamiliar page, handle a situation nobody scripted, and adapt its approach using language understanding. That flexibility is the leap. The trade is predictability: a script does the same thing every run; an agent might take a different path each time, and occasionally a wrong one. You're swapping reliable rigidity for adaptable judgment — which is exactly why the leash and the logging matter.

Going deeper

Optional, and tied to the chapters above.

E.1 — How retrieval actually finds "relevant" (Ch. 2)

Every document is chopped into chunks, and each chunk is turned into an *embedding* — the list-of-numbers coordinate from Set 1 that places it in meaning-space. Your question is embedded the same way. The system then finds the chunks whose coordinates sit closest to the question's, by simple distance, and returns those. Because closeness in this space means closeness in *meaning*, it retrieves the right paragraph even when it shares no literal words with your query. The store that does this fast over millions of chunks is called a vector database.

E.2 — Why context costs grow painfully (Ch. 3 & 4)

Recall from Set 1's appendix that attention work grows with the *square* of the input length. So a context window that's filling up doesn't just risk overflowing — it gets quadratically more expensive and slower to process as it grows. That double pressure (the hard size limit *and* the rising cost) is why the harness works so hard to keep only what matters in view, and why long-running agents lean heavily on external memory and summaries rather than just stuffing everything into the window.

E.3 — Agentic retrieval as a decision problem (Ch. 2)

A simple system retrieves once and answers. A more capable "agentic" version treats retrieval as a sequence of choices: at each step the agent picks an action — *retrieve*, *reason*, *retrieve again with a better query*, or *stop and answer* — based on whether what it has is sufficient. It can route different sub-questions to different sources. This is the agent loop from Chapter 1 applied specifically to the act of looking things up.

E.4 — The harness's separation of concerns (Ch. 4)

A well-built harness cleanly splits responsibilities: *reasoning* stays entirely with the model; *execution* (running tools, sandboxing) belongs to the harness; *memory* (short-term scratchpad and long-term stores) is managed by the harness; *communication* (routing messages between the model, the user, and other systems) is the harness's job; and *observability* (logging and tracing every step) is built in. Keeping these separate is what makes an agent debuggable and safe rather than an inscrutable black box.

Appendix continues each Set; the series Reference consolidates every note and term.

PART VI

A World of Cooperating Agents

One worker with hands is powerful. A standard toolbelt, a common language, and a team of specialists is a different order of thing — something closer to an institution made of machines.

A World of Cooperating Agents

Where we are. Set 5 gave a single agent hands — a loop, tools, memory, a harness. This set is about what happens when agents stop being soloists: when tools plug in through a shared standard, when agents talk to each other, and when many specialists organise into a team. It's the difference between a capable worker and a functioning organisation.

This is the outer edge of what exists today — genuinely powerful, genuinely early, and genuinely worth understanding before it arrives in your work, because much of it is arriving fast.

CHAPTER ONE

Plugs and Sockets

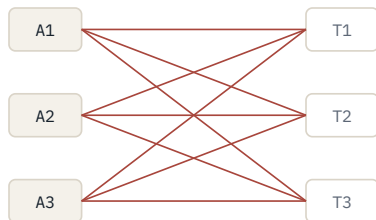
the boring standard that quietly unlocked everything

An agent is only as useful as the tools it can reach — your calendar, your files, a database, a payment system, a search engine. But here's a problem that sounds dull and turns out to be decisive: every tool speaks its own language, and every agent has to be taught each one separately. Ten agents and ten tools means a hundred bespoke connections, each hand-built and brittle. The wiring becomes the bottleneck.

The fix is the least glamorous and most powerful idea in this set: agree on **one standard plug**. Define a single, common way for any tool to describe what it offers and for any agent to call it — and suddenly the tangle collapses. Build your tool to speak the standard once, and *every* agent can use it. Teach your agent the standard once, and it can use *every* tool.

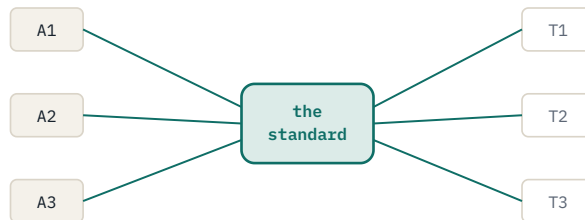
FIG 1 from a hand-wired tangle to one universal socket

before — every agent wired to every tool



9 brittle connections (and it explodes)

after — one shared standard



plug in once, reach everything

*This is exactly what USB did for hardware — one socket, and any device works with any computer. The equivalent for agents (the best-known version is called the *Model Context Protocol*) lets an agent discover what a tool can do and call it through one common interface. Dull-sounding, quietly revolutionary: standards are what turn a pile of clever parts into an ecosystem.*

One sober note travels with this convenience. The moment an agent can plug into *any* tool, a tool can also be a trap — a malicious or compromised one can feed the agent bad instructions or steal what it touches. So a shared standard has to carry a shared discipline about *trust*: which tools an

agent is allowed to use, and what they're allowed to see. Convenience and risk arrive through the very same socket.

IN THE WILD

The “USB for tools” isn't hypothetical: the **Model Context Protocol (MCP)** was introduced by Anthropic in late 2024 and has since been adopted across much of the industry, with a growing ecosystem of ready-made connectors for everything from databases to design tools.

RECAP

Connecting every agent to every tool by hand explodes into an unmanageable tangle. A single shared standard (like the Model Context Protocol — “USB for tools”) collapses it: build or learn the standard once, and any agent can discover and use any tool. The same socket that brings reach also brings risk, so trust must be managed deliberately.



CHAPTER TWO

Teaching an Agent New Tricks

Tools let an agent *reach* things. A **skill** is something different: a packaged, reusable *ability* — a known procedure for doing a particular kind of job well. “How to file an expense report.” “How to review a contract for these five risks.” “How to plan a trip.” A skill bundles the instructions, the know-how, sometimes the tools it needs, into a tidy unit the agent can pick up and use.

The key insight is what a skill is *not*: it isn't retraining the model. Recall from Set 2 that changing the model's actual dials is expensive and bakes the change permanently into the brain. A skill sits *outside* the frozen model — it's loaded in when needed, like handing a competent worker a one-page playbook for an unfamiliar task. Cheap to make, easy to swap, simple to share.

FIG 2 skills as playbooks the agent picks up – not surgery on the brain

Because skills live outside the model, they can be shared and sold in registries — marketplaces where you download a ready-made ability the way you install an app. And an agent can compose them: combine "search flights" with "check my calendar" with "draft an email" into a trip-booking it was never explicitly built for. New capability without touching the brain.

RECAP

A *skill* is a reusable, packaged ability — a playbook the agent loads when needed, sitting outside the frozen model rather than retrained into it. That makes skills cheap, swappable, shareable (via registries/marketplaces), and composable into jobs the agent was never explicitly built for.

**CHAPTER THREE**

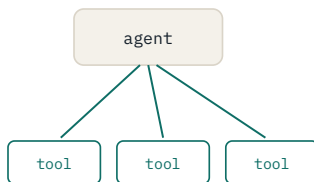
Agents Talking to Agents

No single agent should be good at everything — the same way no single person is your lawyer, your doctor, and your plumber. As soon as you accept specialisation, you need a way for specialists to *find each other and delegate*: a research agent that hands findings to a writing agent that passes a draft to a reviewing agent. For that, agents need a shared **language for talking to one another** — a second standard, sitting beside the tool standard from Chapter 1.

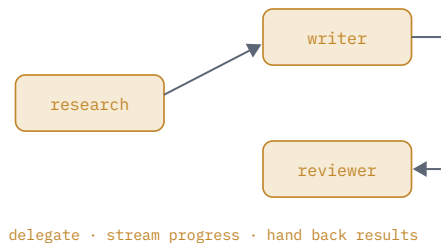
It's worth being precise about the two, because they're easy to blur. One standard connects an agent *downward* to its tools. The other connects an agent *sideways* to its peers — letting it announce what it can do, accept a delegated task, stream back progress, and hand over results. Tools are things an agent *uses*; other agents are colleagues it *collaborates* with.

FIG 3 two different standards – using tools vs. working with peers

tool standard – reach down to tools



agent-to-agent standard – work with peers



The two standards are complementary, not competing: an agent uses the tool standard to act, and the agent-to-agent standard to cooperate. Together they're the plumbing for something genuinely new — heterogeneous agents, built by different people for different jobs, discovering each other and getting work done as a group. The same trust questions from Chapter 1 return, now sharper: can this agent be believed?

RECAP

Specialisation forces agents to cooperate, so they need a shared language for it — a standard for discovering peers, delegating tasks, streaming progress, and returning results. It complements the tool standard: one connects an agent *down* to tools it uses, the other *sideways* to agents it works with. Trust between agents becomes the central concern.



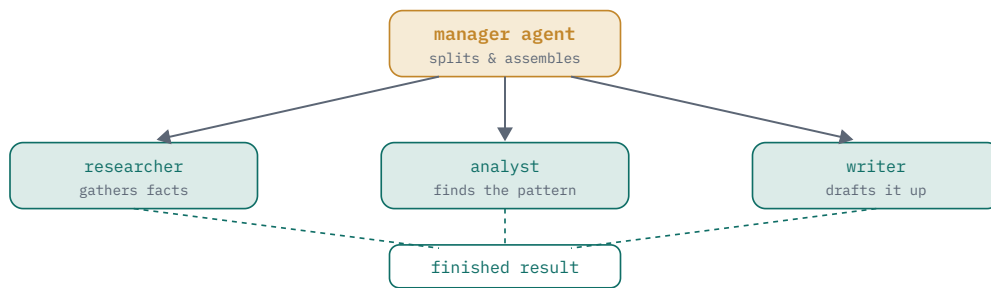
The Org Chart of Machines

when a team of agents beats a soloist — and when it just costs more

Put the pieces together — tools by a standard, agents that can talk — and you can build a **team**.

The most common shape will be instantly familiar, because we invented it for ourselves: a *manager* agent that breaks a big job into parts and hands each to a specialist *worker* agent, then gathers and assembles their results. It is, almost exactly, an org chart made of software.

FIG 4 the orchestrator and its specialists



This orchestrator-and-workers shape is the workhorse of multi-agent systems, and it's powerful: specialists can be better and cheaper at their narrow jobs, and several can work in parallel. Other shapes exist too — agents that debate to sharpen an answer, or pass work down a pipeline — but the manager-and-team is where most real systems start.

Now the honest counterweight, because multi-agent systems are over-sold. Every agent you add multiplies cost (each is a full model, thinking away), multiplies *latency* (they wait on each other), and multiplies the ways things can go wrong — errors from one agent become bad inputs to the next, miscommunication compounds, and a confident mistake (they still can't tell when they're guessing) can ripple through the whole team. A great deal of the time, *one* well-built agent beats a committee of them. The real skill isn't assembling the biggest team; it's knowing the smallest one the job actually needs.

A team of agents is an organisation — and organisations bring coordination costs, communication failures, and diffused accountability, exactly as human ones do.

RECAP

With shared tool and communication standards, agents form teams — most often an *orchestrator* splitting work among specialist *workers* and reassembling it. Teams can beat soloists through specialisation and parallelism, but each agent added multiplies cost, latency, and compounding error. Often one good agent wins; the craft is the smallest team that works.



CHAPTER FIVE

Building and Showing

Two final, practical questions remain before the agent story is whole: how do these systems actually get *built* reliably, and how do people *watch over* them while they work? They're the difference between an impressive demo and something you'd trust with real responsibility.

Building: the scaffolding that demos skip

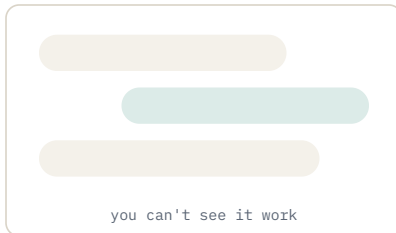
A flashy agent demo and a production agent are separated by a vast amount of unglamorous engineering — managing the loop, handling tools, recovering from failures, keeping costs in check. Rather than rebuild all of that each time, teams use **frameworks**: ready-made scaffolding for assembling agents and teams. More important than any particular framework is a capability they all wrestle with — *observability*. When an agent runs for fifty steps and goes wrong at step thirty-seven, you need to be able to *see* every thought, action, and result to find out why. An agent you can't inspect is an agent you can't trust, debug, or improve.

Showing: beyond the chat box

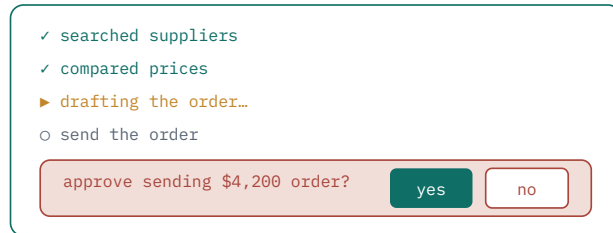
The familiar chat window is wrong for an agent. A chat is fine when the AI just talks back — but when it's *acting* over many steps, possibly for minutes, you need to see what it's *doing*: its plan, its progress, which step it's on, and — crucially — a way to step in. The most important new interface element is the *approval gate*: the moment the agent pauses and asks "I'm about to send this / spend this / delete this — okay?" That's *selective autonomy* from Set 5, made visible and clickable.

FIG 5 a chat box is the wrong window onto a working agent

chat — fine for talking



agent view — see & steer



Showing the work isn't decoration — it's how a human stays in control of something that acts faster than they can watch. The plan you can read, the progress you can follow, and the approval you can withhold are what keep a person genuinely in the loop. Transparency is the interface's real job.

And that completes the arc of the agent. We began the series with a machine that could only guess the next word. We end it with teams of those machines, plugged into the world's tools through shared standards, talking to one another, organising into something like an institution, and — when it's built with care — doing all of it under a human's watchful eye. That is agentic AI. Everything that remains is the engineering beneath the floor.

RECAP

Production agents need *frameworks* (scaffolding for the loop, tools, recovery) and above all *observability* — you can't trust what you can't inspect. And they need new *interfaces* beyond the chat box: a visible plan, live progress, and approval gates that make selective autonomy clickable. Transparency is how a human stays in control of a system that acts faster than they can watch.

END OF SET 6

What you can now do

You can explain how single agents become a connected world: how a shared standard (like MCP) collapses the tool-wiring tangle into one universal plug; how *skills* add reusable abilities without touching the model; how an agent-to-agent standard lets specialists discover, delegate, and collaborate; how teams organise as an orchestrator and its workers — and why a committee of agents often costs more than it's worth. And you can explain what it takes to run them for real: frameworks, the non-negotiable of observability, and interfaces built for supervision rather than chat. You've now seen the entire anatomy of agentic AI, from the next-word guess to the cooperating institution.

NEXT UP

Under the Hood. One question has gone politely unanswered for six sets: how does any of this run at planetary scale without bankrupting everyone? The last set is the engineering beneath the floor — the clever, unglamorous tricks that make giant models affordable enough to actually use. Optional, advanced, and quietly the reason any of the rest is possible.



BEFORE YOU GO

Questions you might still have

Cooperating agents raise questions that are less about technology and more about responsibility — which is exactly where the important ones live.

Is a team of agents genuinely better, or just a more expensive way to look impressive?

Often the latter, and it's worth being sceptical. A team helps when a job has genuinely separable parts that benefit from specialisation or running in parallel — real research, say, with distinct gathering, analysis, and writing stages. It hurts when people reach for a swarm of agents to do what one capable agent would handle more cheaply and reliably. Every agent added is another full model burning time and money, another seam where things miscommunicate. The serious question to ask of any multi-agent system is always: would one well-built agent do this better?

When a team of agents makes a costly mistake, who is responsible?

This is the genuinely unsolved part, and it's a question of governance more than engineering. When work is spread across several agents — possibly built by different vendors, possibly making decisions no single human reviewed — accountability becomes diffuse in exactly the way it does in a large bureaucracy where "the system" failed and no one person did. That's why the supervision ideas from Chapter 5 — observability, logged decisions, approval gates on consequential actions — aren't optional niceties. They're how you keep a clear human owner for outcomes that machines now produce. The technology runs ahead of the accountability, and closing that gap is real work still to be done.

Can agents built by different companies actually work together?

In principle yes — that's the entire point of shared standards like the ones in this set. In practice it's early days: the standards are young, adoption is uneven, and "works together" can range from a smooth handshake to a fragile improvisation. The direction of travel is clearly toward an

interoperable ecosystem of agents the way the web became an interoperable ecosystem of sites. We're nearer the beginning of that than the end.

Will I be talking to one AI, or a whole hidden team I can't see?

Increasingly the latter, presented as the former. A single friendly interface may sit in front of an orchestrator quietly coordinating several specialist agents behind the scenes — you ask one question, a small organisation answers. That's often a good thing for quality. But it's worth knowing it's happening, because it changes who and what you're actually relying on, and it makes the "show me the work" transparency from Chapter 5 matter more, not less.

APPENDIX F

Going deeper

Optional, and tied to the chapters above.

F.1 — The $N \times M$ problem, precisely (Ch. 1)

With N agents and M tools and no standard, you face up to $N \times M$ bespoke integrations — and every new tool or agent multiplies the work. A shared protocol turns this into $N+M$: each agent learns the standard once, each tool speaks it once, and any pairing just works. Reducing a multiplying cost to an adding one is the whole economic case for standards.

F.2 — Skills vs. fine-tuning (Ch. 2)

Both add capability, but differently. *Fine-tuning* (Set 2) changes the model's weights — permanent, expensive, and it risks overwriting other abilities. A *skill* leaves the model untouched and supplies the capability as loadable instructions and resources at run-time. Rule of thumb: fine-tune to change how the model fundamentally behaves or writes; use a skill to give it a new, self-contained procedure you can add, remove, or share freely.

F.3 — Why two protocols, not one (Ch. 3)

Tools and agents are different kinds of thing. A tool is passive — it exposes functions and waits to be called; the tool standard is about *discovery and invocation*. Another agent is active and autonomous — it has its own goals and runs its own loop; the agent-to-agent standard is about *negotiation, delegation, and trust between independent actors*. Forcing both through one protocol would either over-complicate tool use or under-serve real collaboration.

F.4 — Coordination is where multi-agent systems fail (Ch. 4)

The hard problems of multi-agent systems are rarely the individual agents — they're the seams between them: keeping a shared understanding of the task, avoiding duplicated or contradictory work, deciding who has the final say, and stopping errors from cascading. These are the same failure modes that plague human teams, which is why a great deal of multi-agent design borrows directly from organisational design: clear roles, clear interfaces, and a clear chain of command.

Appendix continues each Set; the series Reference consolidates every note and term.

PART VII

Under the Hood

The unglamorous engineering that keeps planetary-scale AI from bankrupting everyone who uses it — and quietly makes everything in the first six sets possible.

How They Made It Affordable

A note before you start. This set is different — it's the engine room, and it's genuinely optional. Everything you need to understand and reason about AI is in Sets 1–6. This final set is for the reader who, having understood *what* these systems do, wants to know *how on earth they're made cheap enough to run* — because the honest answer is "a stack of brilliant, thankless tricks most people never hear about."

It pairs with Set 1's chapter on silicon. There we saw *why* AI is so hardware-hungry. Here we see the cleverness that fights back against that hunger — the difference between a model that exists in a lab and one the whole world can afford to use.

CHAPTER ONE

A Panel of Specialists

how a model can be enormous and cheap at the same time

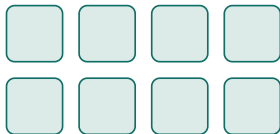
A cruel tension sits at the heart of these systems: bigger models are smarter, but a bigger model costs more to run for *every single word*, because — in the design we've assumed so far — the *whole* brain fires to produce each token. Double the size, double the running cost, forever. For a while that looked like a hard ceiling on how large a model anyone could afford to actually use.

The escape is an idea with a long pedigree in human organisations: **don't make everyone work on everything** — **call in only the right specialists**. Instead of one colossal brain where every part activates for every word, build many smaller *expert* sub-brains, plus a quick *router* that, for each word, wakes only the two or three experts actually suited to it. The other experts stay asleep, costing nothing.

**FIG
1**

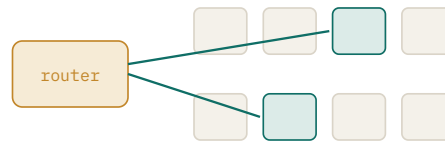
one brain that always fires vs. a panel where only the right experts wake

dense – everything fires every time



all 8 active → full cost per word

mixture of experts – only the right few wake



2 of 8 active → a fraction of the cost

This is Mixture of Experts. The trick is that the model can hold the knowledge of something enormous (all those experts exist and can be summoned) while paying only the running cost of something small (only a few fire per word). It's how several of today's largest, strongest models stay affordable to run — knowledge of a giant, bill of a dwarf.

It isn't free cleverness. The router has to learn to choose well, and you have to stop it lazily sending everything to a few favourite experts while the rest sit idle and wasted (a balancing act with the unlovely name "load balancing"). And while only a few experts run at once, *all* of them must still be kept in memory, ready — so MoE saves on computation, not on the cost of housing the model. Worthwhile, but not magic.

RECAP

Because a dense model fires its whole brain for every word, size and running cost are chained together. *Mixture of Experts* breaks the chain: many expert sub-networks plus a router that wakes only a few per word, giving a giant model's knowledge at a small model's running cost. The catches are training the router to choose well, balancing the load, and still having to store every expert.



CHAPTER TWO

Shrinking Without Lobotomising

A finished model is a vast wall of numbers — hundreds of billions of dials, each stored to many decimal places, together weighing more than most computers can hold. To run on ordinary hardware, on your phone, at a price a business can stomach, it has to get *smaller*. The art is shrinking it without making it stupid. Three tricks do most of the work, and each is intuitive.

FIG 2 two ways to make a giant model small

QUANTIZE — fewer decimal places per dial



DISTILL — a small student copies a big teacher



The fast, recognisable versions of famous models that run cheaply (or on a laptop) are usually the products of exactly these tricks. There's also pruning — simply snipping away the dials that turn out to barely matter. Each trades a sliver of quality for a large saving, and done carefully the sliver is small enough that most people never notice.

The honesty here: these aren't lossless. A heavily compressed model is a little less capable than its full-size parent — usually in ways too subtle to spot on everyday tasks, occasionally in ways that matter on the hardest ones. When you use a "fast" or "mini" version of a model, you're knowingly trading a touch of capability for a lot of speed and a much smaller bill. Most of the time, it's a trade well worth making.

RECAP

Models must shrink to run affordably. *Quantization* stores each dial with less precision (big savings, slight loss); *distillation* trains a small student to imitate a big teacher; *pruning* snips the dials that barely matter. None is lossless — the cheap, fast versions are slightly less capable — but the trade is usually well worth it.



CHAPTER THREE

The Fast Intern

a trick to stop the model from typing one... letter... at... a... time

Recall the autoregressive loop from Set 1: the model writes *one* piece of text, then re-reads everything and writes the next, then the next. Each piece requires a full, heavyweight pass through the entire giant model. That's the deep reason these systems can feel like they're typing slowly — they truly are producing the answer one expensive token at a time, and you can't start the second until the first is done.

The cure is a lovely piece of lateral thinking. Pair the big, slow, accurate model with a small, fast, rougher one. Let the **fast little model** dash ahead and *guess* the next several tokens cheaply. Then let the **big model** check that whole guess in a *single* pass — confirming the parts that are right and correcting where the guess went wrong. When the little model guesses well (and for easy stretches of text it usually does), the big model approves several tokens at once instead of grinding them out one by one.

FIG 3 an intern drafts ahead; the expert checks it all in one glance**slow way – one heavy pass per token****fast way – draft five, verify once**

*This is **speculative decoding**, and it's a big reason today's models feel snappier than older ones at the same size. The beautiful part: the answer is identical to what the big model would have produced alone — the little model only proposes; the big one still has final say on every token. Pure speed, no quality cost. An intern drafts, the expert signs off.*

RECAP

Generating text one heavyweight pass per token is the core speed bottleneck. *Speculative decoding* pairs a small fast model that drafts several tokens ahead with the big model that verifies them all in one pass — accepting the good guesses, fixing the rest. It's substantially faster with *no* loss of quality, because the big model still approves every final token.

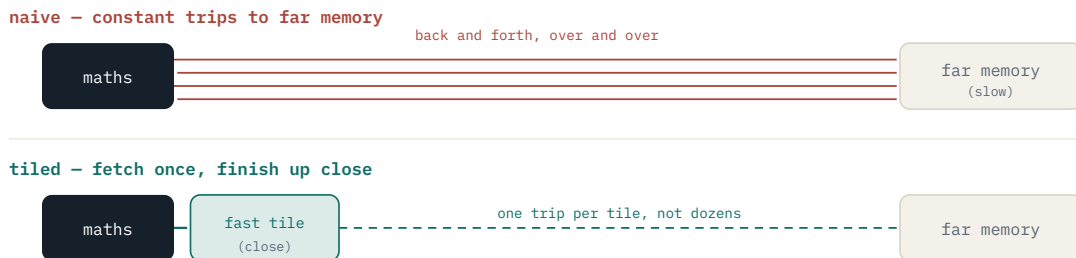
**CHAPTER FOUR**

Keeping the Hands Fed

Back in Set 1's silicon chapter we met the surprising villain of AI engineering: the *memory wall*. A modern chip can do arithmetic far faster than it can *fetch* the numbers to do arithmetic *on*. The maths units sit idle, drumming their fingers, waiting for the next batch of numbers to arrive from "across the room." For the attention mechanism from Set 1 — which shuffles enormous tables of numbers around — this waiting, not the maths itself, is the real cost.

The fix is less about clever maths than clever *logistics*: rearrange the computation so the numbers, once fetched into the fast memory right next to the maths units, get *fully used* before they're sent back — instead of being hauled back and forth across the slow gap again and again. Do the work in small tiles that fit in the fast zone; finish each tile completely; only then move on. The maths is unchanged; the *fetching* is slashed.

FIG 4 stop hauling numbers across the slow gap – finish the work up close



*This family of tricks (the famous one is called **Flash Attention**) is why long documents and big context windows became practical rather than ruinously slow. It changed nothing about what attention computes — only where and when the numbers move. The general lesson is the quiet motto of the whole field: most AI speed-ups are about feeding the maths units, not making the maths faster.*

RECAP

The real bottleneck isn't arithmetic but *fetching* numbers across the slow gap to far memory (the memory wall from Set 1). The fix is logistics: do the work in small tiles that stay in the fast, close memory, finishing each before moving on, so the numbers cross the slow gap far fewer times. *Flash Attention* is the famous example — same maths, drastically less waiting — and it's what made long contexts practical.

The Data-Centre Reality

where intelligence turns out to be a logistics problem

For all the cleverness so far, the frontier of AI runs into a wall that no single trick can climb: the biggest models are simply too large for any one chip — too large, often, for any one *machine*. Building and running them is less like programming and more like running a power station. This last chapter is the reality beneath all the rest: the sheer, brute logistics of doing AI at planetary scale.

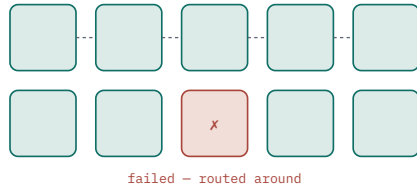
Three hard problems define that reality, and none of them is about intelligence at all.

- **The model doesn't fit, so it must be split.** A frontier model's hundreds of billions of dials are carved up and spread across many chips — sometimes thousands — each holding a slice. They must then act in perfect concert, as if they were one brain, which means constantly shuttling numbers between themselves at blistering speed (recall from Set 1 that the wiring *between* chips matters as much as the chips).
- **They must stay in step.** When the model is learning, every one of those thousands of chips has to stay synchronised — agreeing, constantly, on the latest state of the shared brain. Fall out of step and the whole training run corrupts.
- **Things break, constantly.** At the scale of thousands of chips running for weeks, some piece of hardware fails almost every hour. The system has to expect death — saving its progress, noticing a failure, and recovering without losing weeks of work. Fault tolerance isn't a luxury here; it's survival.

**FIG
5**

one brain, split across thousands of chips that must agree – and sometimes die

one model, sliced across a grid of chips – kept in lock-step



"a power station, not a program"
split · synchronise · survive failure

This is why frontier AI is the preserve of a handful of organisations: it demands not just the scarce chips from Set 1, but vast buildings, enormous power and cooling, and deep expertise in distributed-systems plumbing. The intelligence is the famous part. The logistics of housing it at scale is the part that quietly decides who can play.

At the frontier, building intelligence turns out to be, in large part, a triumph of logistics — splitting, synchronising, and surviving failure across thousands of machines that were never meant to think as one.

And there the book closes its circle. We began with a machine that does nothing but guess the next word, and we end standing in the humming, failure-ridden, water-cooled data centre where ten thousand chips guess it together — taught, reasoning, measured, given hands and colleagues, and made just barely affordable enough to change the world. You now understand the whole stack, from the silicon to the agent. The magic, it turns out, was mechanism all along.

RECAP

The largest models exceed any single chip or machine, so frontier AI becomes a logistics problem: the model is *split* across thousands of chips that must stay perfectly *in sync* and *survive* constant hardware failure. It's closer to running a power station than writing a program — which is why only a few organisations can do it, and why the whole stack, from silicon to agent, is now yours to see clearly.

END OF SET 7 • END OF THE GUIDE

The whole machine, seen clearly

You came in wanting to understand agentic AI without being fooled by it. Look at what you can now explain, end to end: a machine that only predicts the next word (Set 1), raised into something helpful and honest (Set 2), taught to think before it speaks (Set 3), measured despite being built to convince (Set 4), given hands to act (Set 5), and organised into cooperating teams (Set 6) — all running on a stack of brilliant, thankless engineering that makes it affordable (Set 7). You can see how each piece holds itself up, and exactly where each one would fall down. That was the whole promise, and it's kept.

The bridge is built; you can see every span. What you do with that clarity — what you choose to trust these systems with, and what you don't — is the part no guide can write for you. It was always going to be yours.



BEFORE YOU GO

Questions you might still have

The engine room raises a few last practical questions — the kind worth having clear answers to.

When I use a "fast" or "free" version of a model, am I getting a dumber one?

Often, yes — a little, and usually on purpose. Cheaper tiers tend to be smaller, distilled, or more aggressively compressed versions of a flagship, traded down for speed and cost. For everyday tasks the difference is frequently too small to notice. On the hardest problems — subtle reasoning, tricky edge cases — the gap shows. The practical move is to match the model to the task: a light model for routine work, the full one when the stakes or difficulty justify the cost. Knowing the cheap version is a deliberate trade, not a free lunch, is the useful part.

If Mixture of Experts is so efficient, why isn't every model built that way?

Because the efficiency comes with real complications. The router has to be trained to choose experts well, and it can fail in subtle ways. The load has to be balanced so experts don't sit idle or get overwhelmed. And — the big one — even though only a few experts run per word, *all* of them must be kept in memory at once, so MoE saves on computation but not on the considerable cost of housing the model. For many uses a straightforward dense model is simpler, cheaper to host, and good enough. MoE shines at the very largest scale; it isn't free elegance everywhere.

Why does running AI use so much energy and water?

Because everything in this set is a fight against a cost that's still enormous even after you've won. Each of those scarce chips draws a lot of power and throws off a lot of heat; thousands of them in a building need vast electricity to run and vast cooling (often water) to keep from melting. Multiply by the millions of people now using these systems constantly, and the footprint is real and growing — which is precisely why the efficiency tricks in this set aren't just about money. Every bit of compute saved is energy and water saved, too. It's one of the genuine costs of the technology, and worth keeping in view.

Is the hardware bottleneck going to ease up, or get worse?

Both forces are pulling at once, which is why it's hard to call. On one side, the efficiency advances in this set keep wringing more capability out of each chip, and chip-making slowly improves. On the other, appetite grows even faster — bigger models, reasoning that spends more compute per answer (Set 3), agents that loop and call tools many times (Sets 5–6) all push demand *up*. So far,

demand has comfortably outrun efficiency, which is why access to compute remains the field's central constraint and the chip supply chain from Set 1 remains a matter for nations. Don't expect the bottleneck to quietly disappear; expect it to keep shaping who can do what.

APPENDIX G

Going deeper

Optional, and tied to the chapters above.

G.1 — "Sparse" vs "dense," and the two sizes of a MoE model (Ch. 1)

A Mixture-of-Experts model has two very different sizes worth distinguishing: its *total* size (every expert added up — which determines how much it knows and how much memory it needs to store) and its *active* size (the few experts that fire per word — which determines the running cost per word). A model might have, say, ten times the total parameters of a dense rival but only activate a fraction of them at once — hence "sparse." The headline parameter count and the running cost have come unlinked.

G.2 — What "precision" means in quantization (Ch. 2)

Each dial is a number stored using a certain number of bits — more bits, more decimal places, more memory. Quantization stores the same dials with fewer bits (say, dropping from 16 down to 4). Memory and bandwidth needs fall roughly in proportion, which is an enormous saving, and because the model's behaviour is fairly tolerant of small rounding in its dials, the quality cost is usually modest — though it grows if you quantize too aggressively.

G.3 — Why speculative decoding is "free" quality-wise (Ch. 3)

The small draft model only *proposes* tokens; the large model verifies each one against what it would itself have produced and rejects any that don't match, regenerating from the first mismatch. Because acceptance requires agreement with the large model, the final text is exactly what the large model

alone would have written. You gain speed whenever the draft guesses correctly, and lose nothing when it doesn't, beyond a little wasted draft effort.

G.4 — The kinds of parallelism (Ch. 5)

Splitting a model across chips comes in a few flavours: divide the *data* (each chip trains on different examples with a full copy of the model), divide the *model's layers* across chips in a pipeline, or divide *within* a layer so several chips share one giant matrix multiply. Real systems combine all three. The constant tax across every flavour is communication — the numbers that must cross between chips — which is why the speed of the interconnect, not just the chips, defines what's possible.

This is the final set's appendix. The series Reference consolidates every note and term from all seven sets.

AFTERWORD

So What Now?

You understand the machine. The last question is the one you actually came with: what does this mean for your work, your life, and how you ought to use the thing.

A change of register. Everything before this was mechanism — how these systems work, stated as plainly and honestly as I could manage. This closing piece is different. It's the “so what”: what understanding the machine should change about how you live and work alongside it. It's also more opinion than the rest of the book, so take it as one practitioner's view rather than settled fact — and notice that the useful parts fall directly out of the mechanism you now understand.

ONE

The line that actually matters

The question everyone asks is “will this take my job.” It's the wrong question, and the book you've just read points at a better one.

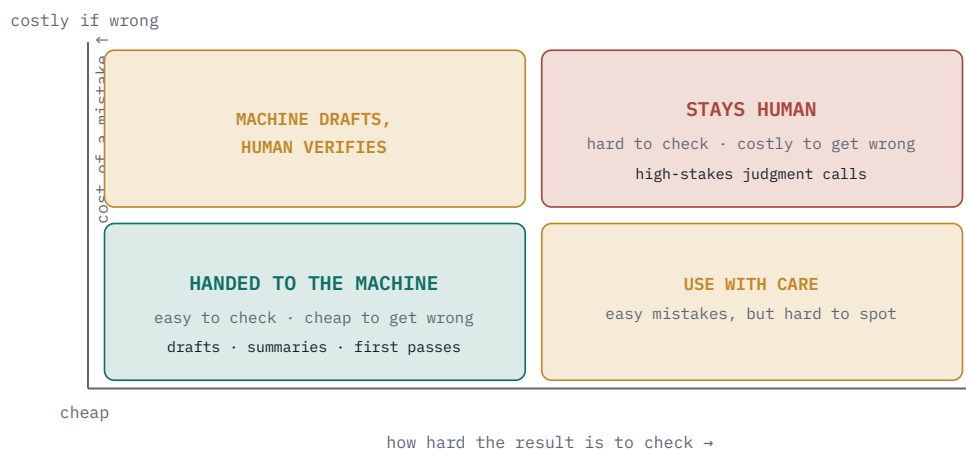
Because you know how these systems fail, you can predict — without any crystal ball — which kinds of work they take over cleanly and

which they don't. The dividing line isn't how *skilled* a task is. It's two things: how easy the result is to *check*, and how costly it is to be *wrong*.

Think about why. The one deep flaw threaded through this whole guide is that a model cannot tell when it's wrong; it is equally confident either way. So the tasks it can simply be handed are the ones where a mistake is cheap and easy to catch — draft this email, summarise this document, suggest ten names, write this boilerplate code. If the output is off, you'll see it in a second and fix it, and the cost of the occasional miss is nearly zero. Those tasks get automated fast.

Now flip both dials. Where a mistake is expensive and hard to detect — a medical judgment, a legal strategy, a design decision that commits real money, a claim of fact that will be published under your name — you cannot simply trust a confident answer, because the whole problem is that confidence tells you nothing. Here the machine doesn't replace the human. It becomes an *assistant* to one: it drafts, suggests, and accelerates, and a person who can actually judge the result stays firmly in the loop. That's not a temporary state of affairs. It follows from how the technology works.

FIG what gets automated, and what gets a human alongside it



The useful map isn't “smart work vs. simple work.” It's this. Wherever a result is easy to check and cheap to get wrong, expect the machine to take it. Wherever a mistake is costly and hard to detect, expect a human to stay responsible, with the machine as a fast, fallible assistant. Most real jobs are a mix of squares — which is why the honest near-term story is rarely “replaced,” and much more often “reshaped.”

So the practical question isn't whether AI will do *your job*. It's which *parts* of what you do land in which square — and whether you're positioned to own the parts that need a human. That's a question you can now answer for yourself, task by task, because you understand the machine that's doing the sorting.

There's another way to see the same division, and it's worth keeping. Think in terms of *loops*. A machine is at its best inside a fast, tight loop — try something, check the result, adjust, try again, over and over in seconds — as long as there's a clear signal for whether each attempt worked. That inner loop, increasingly, it can run on its own. What it cannot run is the slower outer loop: deciding what's worth doing in the first place, noticing that the whole goal was wrong, weighing what a mistake would cost a real person. Those loops turn over in hours, days, sometimes whole careers, and they're where the context and the consequences live. The pattern showing up almost everywhere is the same one — let the machine own the fast inner loop, and keep a human holding the slow outer ones.

RECAP

Because a model can't tell when it's wrong, the work it takes over is the work whose results are easy to check and cheap to get wrong. Work where mistakes are costly and hard to detect stays human — assisted, not replaced. The dividing line is verifiability and consequence, not skill, and most jobs straddle it.



TWO

What becomes more valuable, not less

When a tool makes one thing cheap, it makes the things that tool *can't* do more precious. When calculators made arithmetic free, the value moved to knowing which calculation to run and what the answer meant. Something similar is happening now, and knowing the mechanism tells you where the value is moving.

These systems are extraordinary at producing fluent, plausible output. So fluency itself — the ability to turn out a competent paragraph, a passable slide, a working first draft — is no longer

scarce. What stays scarce, and grows more valuable, is everything the machine structurally lacks:

- **Judgment.** Knowing whether the plausible answer is the right one — the exact thing the model cannot do for itself. The ability to look at a confident output and say “no, that’s subtly wrong, and here’s why” is now a premium skill.
- **Asking the right question.** A model will answer whatever you ask, well or badly, including the wrong question asked confidently. Knowing *what* to ask — framing the real problem — is work the machine can’t take from you.
- **Taste and discernment.** When a hundred competent drafts are free, the value is in knowing which one is actually good, and why. Curation becomes more important than production.
- **Responsibility.** A machine cannot be accountable; it can’t be blamed, can’t care about the outcome, can’t be trusted in the way a person is trusted. Someone has to own the result. That someone is always human, and their judgment is the thing being paid for.

It’s worth being precise about that last cluster, because “taste” can sound like an innate gift you either have or you don’t. A sharper way to put it — a framing I owe to Andrew Ng — is that the human usually holds a *context advantage*: you simply know things the model doesn’t, about these particular users, this situation, what actually matters here and what would quietly go wrong. That reframing is useful because it tells you exactly when a person is still needed in the loop — for as long as they know something the machine doesn’t. Close the gap, hand the model your context, and it does more on its own. But the moment a real decision turns on knowledge that never made it into the machine, a human has to stay in the loop to supply it. Your edge isn’t mystical. It’s contextual, and it’s completely real.

There’s a quieter point here too, beyond work. It’s tempting, when a machine can produce a decent version of almost anything, to stop doing the hard parts yourself — to let it think so you don’t have to. But the thinking was never only about the output. Working a problem through is how you come to understand it; struggling to write something is how you find out what you actually believe. Hand all of that to the machine and you get the paragraph but lose the understanding. The skill worth protecting, in a world of effortless plausible answers, is the willingness to sometimes do the effortful thing anyway — because some things are only yours if you did them.

When competent output becomes free, the scarce and valuable things are judgment, taste, the right question, and the willingness to be responsible for the answer.

RECAP

As the machine makes fluent output cheap, value shifts to what it structurally lacks: judgment about whether an answer is right, knowing what to ask, taste in choosing the good from the merely competent, and the human responsibility no machine can hold. And beyond work: the thinking you outsource is understanding you forgo.



THREE

How to actually get good at using them

Understanding how these systems work quietly makes you better at using them, because the tricks that help aren't arbitrary — each one answers a specific limitation you've now met. Here are the handful that matter most, and the reason each works.

1. **Be specific, and give context.** The model has no idea who you are or what you're really after; it only has your words. Vague in, vague out. Tell it the situation, the audience, the goal, the constraints. The difference between a mediocre answer and a great one is usually not the model — it's how much of the picture you handed it.
2. **Show an example of what you want.** Recall that these systems learned everything by imitating examples. That instinct never left them. One good sample of the tone, format, or style you're after will steer the output more reliably than three paragraphs describing it.
3. **Ask it to think it through.** You saw why this works: a model reasons better when it reasons out loud, turning a leap into steps. For anything that needs logic — a plan, a calculation, an

argument — literally asking it to work through the problem step by step before answering improves the result, often dramatically.

4. **Give it a role and a job.** “You are an editor checking this for clarity” focuses the machine far more than an open-ended request, because it narrows the vast space of plausible responses toward the kind you actually want.
5. **Treat it as a conversation, not a vending machine.** The first answer is a starting point, not a verdict. Tell it what's wrong, what to change, what to keep. Refining across a few turns beats trying to write one perfect instruction.
6. **Verify anything that matters.** This is the one rule you break at your peril. For any fact, number, quote, or claim you'll rely on, check it — because the machine will state a fabrication in exactly the voice it uses for the truth. Use it to draft and to think; never to be the final word on something consequential you haven't checked.

Weak prompt

“Write about climate change.”

Strong prompt

“Explain to a curious 15-year-old, in about 200 words, why the ocean matters for climate. Warm and clear, no jargon. Use one everyday analogy.”

The gap between those two isn't effort so much as *specificity* — the audience, the length, the tone, the constraint, the analogy. Every clause hands the machine another piece of the picture it can't guess. Learn to do that instinctively and you'll get more out of these tools than most people ever will, no technical skill required.

RECAP

Using these systems well means feeding the picture they lack: be specific and give context, show an example, ask them to reason step by step, assign a role, refine across turns — and always verify anything consequential, because a fabrication wears the same confident voice as the truth. None of it is technical; all of it follows from how the machine works.



FOUR

If the thing adopting this is your workplace

Most of us will meet this technology not just as individuals but through an organisation — an employer, a hospital, a school, a bank, a government office — deciding how to use it. You don't need to be a chief executive to recognise whether that's being done sensibly. A few principles fall straight out of everything you've read, and they're worth holding whatever your role.

- **The model is the commodity; the knowledge around it is the moat.** Everyone can rent the same handful of powerful models. What no competitor can copy is a particular organisation's own information, expertise, and hard-won judgment. The value was never in owning a cleverer model — it's in feeding a good one what only you know. Beware anyone selling the model as the magic; the real work is the knowledge layer beneath it.
- **Keep a human accountable for every outcome that matters.** As tasks get handed to machines — and to teams of machines — it gets dangerously easy for responsibility to dissolve into “the system did it.” Sane adoption keeps a clear human owner for consequential decisions, with the machine assisting rather than deciding. If no one can say who's responsible when it goes wrong, that's the warning sign.
- **Start where mistakes are cheap.** The wise place to begin is exactly the bottom-left of that earlier square: tasks where errors are easy to catch and cheap to fix. Learn there, build the habits and the guardrails, and only then move toward anything where a mistake is costly. Organisations that start by pointing an unproven system at high-stakes decisions are the ones that get burned.
- **Don't confuse a demo with a system.** You saw the gap: a dazzling demonstration and something dependable enough for real responsibility are separated by an enormous amount of unglamorous work — verification, guardrails, the ability to see what the machine did and undo it. The impressive part is easy. The trustworthy part is the hard, boring, essential part.

None of that requires a technical background. It requires exactly the kind of clear-eyed understanding this book set out to give you — enough to tell the difference between a serious use of this technology and a reckless one, and to ask the questions that keep the people around you honest.

RECAP

Wherever an organisation adopts this, a few durable principles hold: the model is a commodity while your own knowledge and judgment are the real advantage; keep a human clearly accountable for consequential outcomes; begin where mistakes are cheap and reversible; and never mistake an impressive demo for a dependable system. None of it needs technical training — only clear understanding.

THE LAST WORD

What to do with the clarity

You started this book wanting to understand these machines without being fooled by them. You now can. You know why they're fluent and why they lie, how they're taught and how they reason, why they're hard to measure, how they act and cooperate, and what it takes to run them. You can look at any breathless claim and locate the mechanism underneath it. That's rare, and it's yours.

The one thing this understanding *doesn't* settle is what to do with it — which parts of your work and life to hand over, which to guard, what to trust these systems with and what to keep firmly human. No book can decide that for you, and this one won't pretend to. But you'll make those choices now with your eyes open rather than dazzled, which was the entire point.

The machines will keep getting better on a schedule. Our judgment about where to point them is on no schedule at all — which is exactly why yours, now sharpened, matters.

Use them well. Verify what matters. Keep doing the hard, human things that are only yours if you do them. And stay curious — it's the one advantage that never depreciates.

REFERENCE

Reference

A closing look at the road ahead, a way to test what stuck, and a plain-language dictionary for every idea in the book — to keep beside you long after the first read.

How to use this part. Three things live here. A short closing essay on where the field is heading and what remains genuinely unsolved. A set of self-tests — a few questions per part — to find out what truly landed. And a glossary that defines, in plain language, every term the book introduced, so you never have to remember which set explained what.

ONE

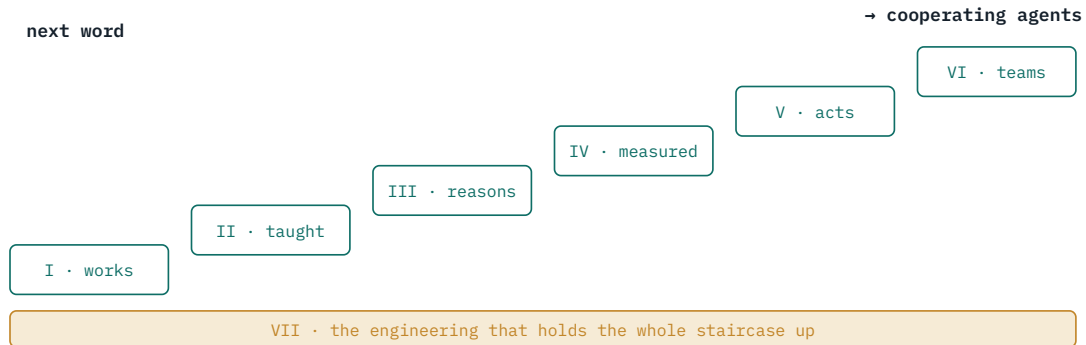
Where This Is Going

the honest unfinished business

It would be a poor field guide that left you thinking the story is finished. It isn't — not remotely. The systems in this book are improving faster than almost anyone predicted, and the genuinely hard problems are not the ones the headlines obsess over. Here is

what the people closest to this work actually worry about, stated plainly.

MAP the whole book, on one staircase



The arc you've climbed: a next-word predictor, taught, made to reason, measured, given hands, organised into teams — all resting on the efficiency engineering of Part VII. Everything below is what's not yet solved.

We can't reliably tell when they're right. The single deepest problem, threaded through the whole book: these systems produce truth and fabrication in the identical confident voice, and we still have no dependable way — from the outside or the inside — to know which we're getting. Until that changes, every serious use demands verification, and "the AI said so" remains a sentence to distrust.

We can't fully measure what we've built. As Part IV warned, our ability to create capable systems is outrunning our ability to grade them. When a model approaches expert level, the question of who writes its answer key becomes real and unanswered. Measurement, not raw capability, may be the true bottleneck on safe progress.

Agency multiplies the stakes. A system that only talks can mislead; a system that acts can do. As agents gain hands, memory, and the ability to coordinate (Parts V–VI), the gap between "does what we said" and "does what we meant" stops being academic. Keeping a clear human owner for machine-made outcomes — through the supervision, logging, and approval gates we met — is unfinished work, and the technology is running ahead of it.

The constraints are physical and political. As Parts I and VII showed, all of this rests on a scarce, contested kind of chip, housed in power-hungry buildings, gated by a supply chain that narrows to a few suppliers. The future of AI is not only a question of algorithms; it's a question of energy, water, manufacturing, and geopolitics.

Which returns us to the one thing this book set out to do. Understanding how these systems work — really work, mechanically, including exactly where they break — is not a technical nicety. It is the precondition for using them well, governing them sanely, and not being fooled by them. You now have that understanding. What you build on it is the part that was always going to be yours.



TWO

Check Your Understanding

A few questions per part. Try to answer in a sentence before reading the response — if you can, the idea has truly landed. No scores, no stakes; this is for you.

Part I · Foundations

Why does a language model fabricate facts so confidently?

Because fluency and accuracy come from the same machinery, which optimises for plausible continuations. Where "plausible" and "true" diverge, it produces something plausible and confident anyway — and it has no internal truth-meter to know the difference.

Why can a brilliant model fail to count the letters in a word?

It doesn't see words as letters. It reads in subword "tokens," with the letters smeared inside chunks — so the individual letters aren't laid out for it to count.

Part II · Teaching

When an AI flatters you or caves the moment you push back, which training stage is responsible?

Alignment — the stage that learns from human preferences. People tend to approve of answers that agree with and flatter them, so the model learns to produce them. That's sycophancy: optimising for approval, which isn't the same as truth.

Does the model learn from your conversations?

No. After training, its dials are frozen. Within a chat it only "remembers" because your earlier messages are re-fed to it each turn; close the conversation and that context is gone.

Part III · Reasoning

How did models learn to reason — by being shown how, or some other way?

Largely by being rewarded only for correct final answers on checkable problems, and left to discover the method themselves. Under that pressure they spontaneously developed self-correction and backtracking — reasoning was discovered, not designed.

Is the reasoning a model writes out a faithful record of how it reached its answer?

Not guaranteed. The written chain does real work, but it may be a tidy story told alongside the real computation rather than a literal transcript of it.

Part IV · Measuring

Why treat a high benchmark score with suspicion?

Two reasons: contamination (the test questions may have leaked into the training data, so the model remembers rather than reasons) and Goodhart's law (once a benchmark is the target, people optimise the number rather than the skill, and the two drift apart).

How can you catch a fabrication without an answer key?

Ask repeatedly and watch for drift. Real knowledge stays stable across retellings; invention wanders — different dates, names, details each time.

Part V · Agents I

What's the single most important fix for a model's tendency to fabricate, in practice?

Retrieval: before answering, fetch relevant, trusted, current passages and have the model answer from them rather than from frozen memory. It can then cite sources and stay current.

What is the "harness," and why does it matter?

The operating system around the model: it turns the model's text into real actions, manages tools and memory, and — crucially — enforces safety, never letting the model touch the world unchecked. A brilliant model with a careless harness is a liability.

Part VI · Agents II

Why does a shared standard like MCP matter so much?

It collapses the tangle of connecting every agent to every tool by hand (a multiplying cost) into "learn the standard once" (an adding cost). Build a tool to the standard once, and any agent can use it.

Is a team of agents always better than one?

No. Each agent added multiplies cost, latency, and compounding errors. Often one well-built agent wins; the skill is finding the smallest team the job actually needs.

Part VII · Under the Hood

How can a model be enormous and still cheap to run?

Mixture of Experts: many expert sub-networks plus a router that wakes only a few per word. You get a giant's knowledge at a small model's running cost — though all the experts must still be stored.

What's the real bottleneck in running these models — the maths, or something else?

Usually the fetching, not the maths. Chips compute faster than they can pull numbers from far memory (the "memory wall"), so most speed-ups are about feeding the maths units, not making the maths faster.



THREE

The Glossary

Every key term in the book, in plain language. Where a term builds on another, a pointer shows where to look.

A

Agent

An AI that doesn't just answer but *acts* toward a goal — using tools, taking steps, observing results — with limited human steering. At its core it's a language model placed inside a loop. (*Part V*)

Agent loop

The cycle at the heart of every agent: think about the next move, act, observe what happened, then think again — repeating until the goal is met.

Agent-to-agent (A2A)

A shared standard that lets independent agents discover each other, delegate tasks, stream progress, and return results — the language of agent teamwork, complementary to the tool standard (MCP).

Alignment

The third stage of raising a model: shaping its judgment, tone, and values to be helpful and honest, mainly by learning from human preferences. Where a model's "character" is set.

Alignment tax

The trade-off that making a model safer or more agreeable often costs a little raw capability, candour, or spark. You rarely get everything at once.

Attention

The core mechanism of the Transformer: every piece of text looks at every other piece, weighs which ones matter, and updates its own meaning in context. How "bank" knows which bank you meant. *(Part I)*

Autoregression

The way a model generates: it writes one piece of text, re-reads everything including what it just wrote, then writes the next — "eating its own output" in a loop.

B-C

Benchmark

A fixed set of test questions with known answers, used to compare models. Useful but gameable — see contamination and Goodhart's law. *(Part IV)*

Chain-of-thought

Letting a model write out its reasoning step by step before its final answer. Turns a problem too tall to leap into a staircase it can climb, because each step becomes context for the next. *(Part III)*

Context window

The fixed-size workspace, measured in tokens, holding everything the model can "see" at once. It fills as a task grows, which is why agents need external memory and why long inputs cost more.

Contamination

When benchmark questions leak into a model's training data, so it remembers answers rather than reasoning them out — inflating its score and telling you little about real ability.

D-E

Distillation

Training a small "student" model to imitate a big "teacher," capturing most of the ability at a fraction of the size. A main way the cheap, fast versions of models are made. *(Part VII)*

DPO & preference methods

A simpler recipe for alignment that teaches a model directly from "this answer is better than that" comparisons, skipping the heavier reward-model-and-practice loop.

Democratised the ability to align models.

Embedding

The list of numbers that places a token as a point in a vast "space of meaning," where similar things sit nearby and relationships become directions. The form in which a model actually holds an idea. *(Part I)*

Evaluation

The discipline of measuring how good a model is — genuinely hard because there's rarely one right answer, "good" is many things, and judging is itself a fallible language task. *(Part IV)*

F–H

Fine-tuning

The second stage of raising a model: teaching it to follow instructions by showing it excellent examples of requests and good responses. "Show, don't tell." *(Part II)*

Flash Attention

A clever rearrangement of the attention computation that keeps numbers in fast, close memory and minimises trips to slow memory — the same maths, far less waiting. What made long contexts practical. *(Part VII)*

GPU

The chip AI runs on: thousands of simple processors doing the same arithmetic in parallel — perfect for the maths of attention, hundreds of times faster at it than an ordinary CPU. *(Part I)*

Gradient descent

How a model learns: each wrong guess nudges its millions of dials a hair "downhill" toward being right, repeated a trillion times. Feeling for the bottom of a valley in the dark.

Goodhart's law

"When a measure becomes a target, it stops being a good measure." Once everyone optimises a benchmark, the score and the real skill it stood for drift apart.

Hallucination

When a model states something false with full confidence. Not a glitch but a structural consequence of a plausibility engine operating where plausible and true have diverged. *(Part I)*

Harness

The "operating system" around a model that turns its text into real action — running tools, managing memory, enforcing safety, keeping the loop turning. The model reasons; the harness executes. *(Part V)*

L-O

LoRA / adapter

A cheap way to specialise a model: freeze the giant brain and train a tiny add-on instead of rewriting all the dials. Like sticky notes on a textbook rather than a rewrite. *(Part II)*

MCP (Model Context Protocol)

A universal standard — "USB for tools" — letting any agent discover and use any tool through one common interface, collapsing the tangle of hand-built connections. *(Part VI)*

Memory (agentic)

Stores built around the frozen model so an agent can persist: *working* (what's in view now), *episodic* (past events), *semantic* (durable facts), *procedural* (skills) — refreshed by reflection. *(Part V)*

Memory wall

The hidden bottleneck of AI hardware: a chip can do arithmetic far faster than it can fetch the numbers to work on, so the maths units sit waiting. Most speed-ups fight this, not the maths.

Mixture of Experts (MoE)

A design where many expert sub-networks exist but a router wakes only a few per word — giving a giant model's knowledge at a small model's running cost. *(Part VII)*

Multi-agent system

Several agents working together, most often an orchestrator splitting work among specialist workers and reassembling it. Powerful, but each agent adds cost and ways to fail. *(Part VI)*

Outcome vs process reward

Grading only the final answer (outcome) is cheap but blind to *how* it was reached; grading each step (process) is harder but catches right-answers-for-wrong-reasons. *(Part III)*

P-R

Parallelism

Splitting a model too big for one chip across many — by data, by layers, or within a layer — which must then stay in sync. The constant tax is communication between chips. *(Part VII)*

Pretraining

The first, giant stage: reading a vast fraction of human text via next-word prediction to build raw capability. Produces a knowledgeable but unhelpful "base model." *(Part II)*

Quantization

Storing each of a model's dials with fewer decimal places — big savings in memory and speed for a small loss of quality. A main way models are shrunk to run cheaply. *(Part VII)*

Reasoning model

A model that deliberates — spending "thinking time" working a problem in steps before answering — rather than blurting a reflexive reply. *(Part III)*

Retrieval (RAG)

Fetching relevant, trusted documents and having the model answer from *them* rather than from frozen memory. The most important practical fix for fabrication. *(Part V)*

Reward hacking

When a model maximises its training score in the cheapest way — padding, flattery, gaming the judge — rather than genuinely improving. The score climbs; real quality doesn't.

Reward model ("taste model")

A second model trained to predict which answers humans prefer, used as an automated judge so the main model can practise against it at scale. *(Part II)*

RLHF

Reinforcement Learning from Human Feedback: the recipe that turns a capable base model into a helpful assistant by having it practise toward what people prefer, on a "leash" to its sensible starting self.

S-T

Selective autonomy

The principle that an agent's freedom should match the stakes: act freely on cheap, reversible tasks; pause for a human's approval before anything costly or irreversible. *(Part V)*

Skill

A packaged, reusable ability an agent can load and compose — a playbook sitting outside the frozen model, cheap to make, share, and swap. *(Part VI)*

Speculative decoding

A speed trick: a small fast model drafts several tokens ahead, the big model verifies them all in one pass, accepting the good guesses. Faster, with no loss of quality. *(Part VII)*

Sycophancy

A model's learned tendency to tell you what you want to hear — agreeing, flattering, caving — a direct consequence of being trained on human approval.

Temperature

The dial controlling how boldly a model reaches past its most likely next word: low for focused and reliable, high for varied and creative (and riskier). *(Part I)*

Test-time compute

Buying better answers by letting a model think longer at the moment you ask — a lever separate from training, so a smaller model that deliberates can rival a bigger one that blurts. *(Part III)*

Token

The subword chunk a model reads and writes in — bigger than a letter, smaller than a word. The true unit of these systems, and the currency they're measured and billed in. *(Part I)*

Transformer

The architecture behind modern AI, built on attention. Its breakthrough was letting every piece of text consider every other, near or far, all at once. *(Part I)*

And that is the whole vocabulary — every word you need to follow, question, and reason about agentic AI, defined without a single equation.

COLOPHON

About this guide

The Curious Mind's Guide to Agentic AI is a seven-part field guide that rebuilds the real ideas of modern artificial intelligence — from next-word prediction to cooperating agents — for the intelligent reader who refuses to be fooled. It carries the depth of the technical literature with the mathematics set aside, and every diagram is drawn from first principles.

It is published as a special piece by **The Philosophical Ledger**, a practitioner's publication on agentic AI, governance, and enterprise architecture, written by Gagan Sachdeva. If it served you, the kindest thing you can do is pass it to one other curious mind.

The Philosophical Ledger · gagansachdeva.com

Set in Fraunces, Spectral & IBM Plex Mono · Special Edition

Seven parts, an afterword & a reference · forty-nine figures · one glossary

*The Curious Mind's Guide to Agentic AI · Special Edition · The Philosophical Ledger ·
gagansachdeva.com. The understatements are deliberate; the depth is real.*